

第 14 章 推理系统

14.1 引言

在深入探究 AI 编译原理之后，将进一步迈向一个与日常生活紧密相连的新领域。这个领域无处不在，无论是日常使用的购物应用、观看在线视频的平台，还是钟爱的游戏，它们都与这个领域息息相关。该领域便是推理系统与推理引擎。

那么，推理系统与推理引擎究竟是什么？它们之间又存在着怎样的差异？推理的具体工作流程是怎样的？在实际应用中又该如何操作？这些问题都亟待去解答。本节将围绕推理系统与推理引擎这两个核心概念展开，详细解释它们的内涵与区别。随后，将聚焦于推理引擎，探讨如何将其模型小型化，如何进行离线优化与压缩，并最终探讨推理引擎的部署与运行优化策略。

14.1.1 推理系统概述

在深入探讨推理系统与推理引擎之前，首先需要明确“推理”这一概念。推理，简单来说，就是在利用大批量数据训练好模型的结构和参数后，使用小批量数据进行一次前向传播，从而得到模型输出的过程。

推理系统，是一个专门用于部署人工智能模型，执行推理预测任务的人工智能系统。它类似于传统的 Web 服务或移动端应用系统，但专注于 AI 模型的部署与运行。推理系统会加载模型到内存，并进行版本管理，确保新版本能够顺利上线，旧版本能够安全回滚。通过提供服务接口（如 HTTP、gRPC 等），推理系统使得客户端能够方便地调用模型进行推理预测。同时，推理系统还可以作为一个微服务，在数据中心中与其他微服务协同工作，共同完成复杂的请求处理任务。

推理引擎，是推理系统中的重要组成部分，它主要负责 AI 模型的加载与执行。推理引擎可分为调度与执行两层，聚焦于 Runtime 执行部分和 Kernel 算子内核层，为不同的硬件提供更加高效、快捷的执行引擎。它可以看作是一个基础软件，提供了一组 API，使得开发者能够在特定的加速器平台（如 CPU、GPU 和 NPU）上轻松地进行推理任务。目前市场上已有多种推理引擎，如字节跳动的 LightSeq、Meta AI 的 AITemplate、英伟达的 TensorRT、华为的 MindSpore Lite 和 MindIE、腾讯的 NCNN 等。

1. 模型小型化

在端侧推理引擎中，模型小型化、轻量化是至关重要的环节。由于端侧设备资源有限，执行轻量的模型结构能够确保高效且稳定的推理性能。模型小型化的核心思想在于设计出更为高效的网络计算方式，从而在减少模型参数量的同时，保持网络精度，并进一步提升模型的执行效率。

模型小型化相关内容将在第 15 章详细介绍，具体包括以下几个方面：

- 重点关注模型小型化过程中的关键参数和指标。这些参数和指标不仅有助于评估模型的小型化程度，还能指导如何更有效地进行模型优化。将首先深入探讨模型大小（通常以参数量来衡量）、计算复杂度（如 FLOPs）等指标，并分析它们之间的权衡关系。

- 介绍一些在模型小型化领域取得显著成果的主干网络（Backbone）或 SOTA（State of the Art）网络模型。这些模型采用创新的网络结构和优化策略，实现了在保证精度的同时，大幅减少模型参数量，降低计算复杂度。将详细分析这些模型的设计思路、网络结构，为读者提供宝贵的参考和启示。

- 重点关注卷积神经网络（CNN）结构下的小型化工作。CNN 是计算机视觉领域中最常用的网络结构之一，对其开展小型化研究具有广泛的应用价值。将介绍一些针对 CNN 的小型化技术和方法，包括轻量级卷积核设计、网络剪枝、量化等方法，并分析它们在减少模型大小和提高推理速度方面的实际效果。

- 简要介绍 Transformer 结构下的小型化工作。Transformer 在自然语言处理领域取得了巨大成功，对其开展小型化研究同样具有重要意义。将探讨一些针对 Transformer 的小型化策略，如采用更高效的自注意力机制、压缩嵌入层等。

通过学习，读者将深入了解模型小型化的重要性、关键指标，以及常见的小型化技术和方法，这将有助于读者在实际应用中更好地进行模型优化和推理性能提升。

2. 离线优化压缩

推理系统作为类似于传统 Web 服务的存在，需要高效响应用户请求并维持高标准的服务等级协议，如响应时间低于 100 ms 等。为了实现这一目标，离线优化压缩在端侧推理引擎中发挥着至关重要的作用。与轻量化网络模型设计不同，离线优化压缩主要通过对轻量化或非轻量化模型应用剪枝、蒸馏、量化等压缩算法和手段，使模型体积更小、更轻便，从而提高执行效率。

第 16 章将围绕离线优化压缩展开详细介绍。

- 首先，探讨低比特量化。低比特量化是一种将模型权重和激活值从浮点数转换为低比特整数（如 8 位、4 位，甚至更低）的技术。通过减少表示每个数值所需的比特数，可以显著减少模型的大小和内存占用，同时加速推理过程。然而，低比特量化也可能导致精度损失，因此需要在压缩率和精度之间找到平衡。

- 接下来，介绍二值化网络。二值化网络是一种极端的量化方法，它将模型权重和激活值

限制为两个可能的值（通常是+1 和-1）。这种方法可以进一步减小模型大小并提高推理速度，但可能导致更大的精度损失。因此，在设计二值化网络时，需要精心选择网络结构和训练策略，以在保持精度的同时实现高效的压缩。

- 除了低比特量化和二值化，模型剪枝也是一种常用的压缩方法。模型剪枝通过移除网络中的冗余连接或神经元来减小模型大小。这可以通过设定阈值来删除权重较小的连接或神经元实现。剪枝后的模型不仅更小，而且往往具有更快的推理速度。然而，剪枝过程需要谨慎处理，以避免过度剪枝导致精度大幅下降。

- 最后介绍知识蒸馏。知识蒸馏是一种将大型教师模型的知识转移到小型学生模型中的技术。通过让教师模型指导学生模型的学习过程，可以在保持较高精度的同时实现模型的小型化。这种方法的关键在于设计有效的蒸馏策略，以确保学生模型能够充分吸收教师模型的知识。

在实际应用中，这些优化压缩方法通常需要根据具体任务和模型特点进行选择 and 调整。通过综合运用这些方法，可以在满足服务需要的同时，实现模型的高效推理和部署。

3. 在线部署和优化

推理引擎的在线部署和优化是确保 AI 模型能够在实际应用中高效运行的关键环节。在模型部署的过程中，推理引擎需要应对多种挑战，包括适配多样的 AI 框架、处理不同部署硬件的兼容性问题，以及实现持续集成和持续部署的模型上线发布等软件工程问题。为了应对这些挑战，推理引擎的在线部署和优化显得尤为重要。

首先，推理引擎需要支持不同 AI 框架训练得到的模型的转换。由于市场上存在多种 AI 框架，如 MindSpore、PyTorch 等，每种框架都有其独特的模型格式和存储方式。因此，推理引擎需要具备模型格式的解析和转换能力，确保不同框架下的模型能够统一地部署到推理引擎中。相关内容将在第 17 章介绍。

其次，推理引擎需要对转换后的模型进行计算图的优化。计算图优化是提升模型推理效率的关键步骤。通过算子融合、算子替换、布局调整、内存分配等方式，可以减少计算冗余、优化内存访问、提高计算并行度，从而显著提升模型的推理速度。相关内容将在第 18 章介绍。

最后，第 19 章对推理引擎的 Kernel 优化方面做了细致的介绍。卷积 Kernel 算子的优化是一个重要的方向，卷积操作是神经网络模型中计算密集且耗时的部分，因此对其进行优化能够显著提升推理性能。

14.1.2 推理应用

本小节将通过具体实例来展示推理系统的实际应用，包括人脸 Landmark 的应用以及利用华为 HMS Core 实现的人脸和手势检测等端侧应用。同时，也将探讨维护推理系统所面临的问题和挑战。

1. 人脸 Landmark

人脸 Landmark 识别技术通过先进的算法技术，能够迅速捕捉并准确识别拍摄者脸部的轮廓、五官位置等关键面部信息。这些信息被实时处理并以一种直观且易于理解的方式显示出来，使用户能够清晰地看到自己脸部的各个特征点。

湖中树（2023）在 CSDN 的文章《Tengine-Kit 人脸检测及关键点》中给出了人脸 Landmark 的示例和终端实现，其具有高度的准确性和实时性，它使得用户可以方便地获取自己的面部信息，并在多种场景下进行扩展应用，如美妆试妆、虚拟形象创建、人脸特效等。随着技术的不断进步和应用场景的不断拓展，人脸 Landmark 识别技术将在未来发挥更加广泛和重要的作用。

2. 人脸检测与手势识别

timer（2020）在文章《Crazy Rockets-教你如何集成华为 HMS ML Kit 人脸检测和手势识别打造爆款小游戏》中展示了在移动终端上应用推理系统的实际案例，通过华为 HMS Core 实现的人脸检测和手势检测功能，为用户带来了新颖而富有互动性的体验。

与人脸检测类似，华为 HMS Core 实现手势检测也是通过推理系统来实时捕捉和识别用户的手势。不同的是，这次是将手势的坐标信息用于控制游戏中的飞船移动。用户可以通过不同的手势来执行不同的动作，如前进、发射导弹等，从而实现更加自然和直观的游戏控制。这种手势控制的方式不仅使得游戏操作更加便捷，还带来了更加丰富和多样的交互体验。

3. 人工客服应用

推理引擎或推理系统在人工客服和人工智能对话方面有广泛应用。以下是一些相关内容。

（1）智能客服：推理引擎可以用于实现智能客服系统，能够理解用户的问题并提供准确的答案。通过对大量的语料库和知识库进行训练，推理引擎可以学习到不同的问题模式和解决方案，从而能够快速准确地回答用户的问题。

（2）对话管理：在人工智能对话中，推理引擎可以帮助系统理解用户的意图和需求，并根据这些信息来引导对话的流向。它可以根据用户的输入和历史对话记录，预测用户可能的问题和需求，并提供相应的回答和建议。

14.1.3 推理系统思考点

在实际维护推理系统的过程中，需要全面考虑并解决以下问题：

（1）如何设计并生成用户友好、易于调用的 API 接口，以使用户能够便捷地与推理系统进行交互。

（2）关于数据的生成，需要明确数据的来源、生成方式以及质量保障措施，确保推理系统能够依赖准确、可靠的数据进行运算。

(3) 在网络环境的影响下, 如何实现低时延的用户反馈是一个关键挑战。需要优化网络传输机制, 减少数据传输的时延, 确保用户能够及时获得推理结果。同时, 需要深入研究智能终端硬件的资源, 合理进行资源分配和加速, 提高推理的运算速度和效率。

(4) 当用户访问量增大时, 如何确保服务的稳定性和流畅性是一个必须面对的问题。需要设计合理的负载均衡策略, 优化系统架构, 提高系统的并发处理能力。此外, 为了应对潜在的风险和故障, 需要制定冗余措施和扩容方案, 确保在突发情况下推理系统能够稳定运行。

(5) 随着技术的不断发展, 未来可能会有新的网络模型上线。需要考虑如何平滑地集成这些新模型, 并制定 AB 测试策略, 以评估新模型的性能和效果。

总之, 维护推理系统需要综合考虑多个方面的问题, 从 API 接口设计、数据生成、网络时延优化、硬件加速资源利用、服务稳定性和流畅性保障、冗余措施与扩容方案, 到新模型上线与测试等方面, 都需要进行深入研究与精心规划。

小结与思考

- 推理系统与推理引擎基础: 推理系统是专门部署 AI 模型并执行预测任务的系统, 类似于 Web 服务但专注于 AI; 推理引擎则是推理系统中负责模型加载与执行的核心组件, 提供高效快捷的执行环境。
- 推理系统应用与挑战: 推理系统广泛应用于日常生活, 如人脸 Landmark 识别、人脸和手势检测等, 同时面临 API 设计、数据质量、网络时延、硬件加速、服务稳定性等维护挑战。

14.2 推理系统介绍

本节将首先概述训练和推理的基本流程, 随后深入分析训练阶段与推理阶段之间的差异。接着, 将深入探讨推理系统的优化目标以及面临的挑战。最后, 通过比较推理系统与推理引擎的流程结构, 将进一步揭示两者在设计 and 实施时需考虑的关键要素。

1. 推理阶段

在模型的推理阶段, 可以将整个的模型部署在 Web 服务器上或者是 IoT 设备上, 通过对外暴露接口 (例如, http 或 gRPC 等) 接收用户请求或系统调用, 模型通过推理处理完请求后, 返回给用户相应的响应结果, 完成推理任务。

与训练阶段类似, 在推理阶段中需要输入小批量真实世界的的数据样本, 执行前向传播过程通过神经网络模型后, 得到计算输出的预测结果。其中, 这个过程不涉及反向传播, 也就不需要计算梯度来更新模型的权重。

同时, 因为推理过程通常是在生产环境中执行的, 因此性能和效率至关重要。因此, 需要优化模型的计算速度和内存占用, 以便实时或高吞吐量地处理输入数据。而测试则是评估模型性能的过程, 通常包括在一组已知的测试数据上运行训练好的模型, 并评估其在测试数据上的

表现，以了解模型的准确性、精确度、召回率等性能指标。测试的目的是验证模型是否能够在真实场景下有效地工作，检查模型在未知数据上的泛化能力。

2. 模型部署

推理系统一般可以部署在云侧或者边缘侧。云侧部署的推理系统更像传统 Web 服务，在边缘侧部署的模型更像手机应用和 IoT 应用系统。两者有以下特点：

- 云（Cloud）侧：云侧有更大的算力、内存，而且供电充足，满足模型进行推理的功耗需求。同时与训练平台连接更加紧密，更容易使用最新版本模型，同时安全和隐私更容易保证。相比边缘侧可以达到更高的推理吞吐量。但是用户的请求需要经过网络传输到数据中心并进行返回，同时使用的是服务提供商的软硬件资源。

- 边缘（Edge）侧：边缘侧设备资源更紧张（例如，手机和 IoT 设备），且功耗受电池约束，需要更加在意资源的使用和执行的效率。用户的响应只需要在自身设备完成，且不需消耗服务提供商的资源。

在训练阶段与推理阶段之间需要通过部署的方式将训练好的模型加载到 Web 服务器或 IoT 设备上，对于推理系统中的部署涉及以下多个步骤，确保训练好的模型能够有效地应用于实际场景。

在部署中可能涉及创建 API 接口、配置服务器、设置数据传输和存储等。在部署后，持续监控模型的性能，并根据需要进行优化。这可能包括调整模型参数、更新推理引擎版本、优化硬件资源分配等。值得注意的是，不同的推理引擎和硬件平台可能具有不同的特性和优化方法，因此在实际部署过程中需要根据具体情况进行调整和优化。

14.2.1 推理场景的重点

训练任务通常在数据中心的异构集群管理系统中进行，这些任务类似于传统的批处理任务，需要数小时乃至数天才能完成。训练任务一般要配置较大的批量尺寸，追求较大吞吐量的中心服务器，将模型训练达到指定的准确度或错误率。

而在推理任务方面，以 HMS Core 为例，它采用 MindSpore Lite 作为推理引擎。由于 HMS Core 需要为全球用户提供服务，因此它必须保持 7×24 h 不间断运行，类似于传统的在线服务模式。

此外，由于每天的调用量超过亿次，且需要快速响应每个服务请求，这对参与推理任务的模型提出了极高的要求。这些模型必须具备高效、稳定且准确的推理能力，以确保用户体验和数据处理的实时性。

在深度学习中，推理任务相较于训练任务确实具有一系列独特的新特点与挑战。具体来说，这些特点和挑战包括以下几点。

- **长期运行服务需求：**神经网络模型在推理阶段通常被部署为长期运行的服务，如互联网服务。这类服务对请求的响应有严格的低时延（Low Latency）和高吞吐量（High Throughput）要求。

● **推理资源约束更为苛刻：**与训练阶段相比，推理阶段通常面临更为严格的资源约束，如更小的内存、更低的功耗等。这是因为推理任务通常需要在各种设备上运行，如手机、嵌入式设备等，这些设备的资源远小于数据中心的商用服务器。

● **部署设备型号多样性：**由于神经网络模型需要在各种设备上运行，但是这些设备的型号和配置可能各不相同，为了在各种设备上实现高效的推理，因此需要进行多样化的定制和优化。例如，在服务器端，可以通过 Docker 等容器技术解决环境问题；而在移动端和 IoT 设备端，由于平台、操作系统、芯片和上层软件栈的多样性，需要更为复杂的工具和系统来支持编译和适配。

在推理阶段中，数据经过训练好的网络模型用于发现和预测新输入的信息。在这个阶段，这些输入数据通过更小的批量大小输入网络，且由于有时延的约束，大的批量大小需要所有批量内样本都处理完才能响应，容易造成时延超出约束。

对于推理阶段，性能目标与训练阶段有所不同。为了最大限度地减少网络的端到端响应时间（End to End Response Time），推理通常比训练批量输入更少的输入样本，也就是更小的批量大小，因为依赖推理工作的服务（例如，基于云的图像处理管道）需要尽可能的更快响应，用户不需要让系统累积样本形成更大的批量大小，从而避免了等待几秒钟的响应时间。

类似其他的推理系统，它们通常也遵循类似的步骤和架构原则。确实，在许多场景中，尤其是在需要高吞吐量或低时延的应用中，直接使用 CPU 进行推理可以避免数据在主存与设备内存之间的拷贝开销。此外，操作系统对 CPU 上的任务提供了更成熟的隔离与任务调度管理支持，这有助于确保推理任务的稳定性和效率。

从图 14.2.1 的推理服务系统架构图中，可以清晰地看到推理服务系统的流程。首先，经过训练后的模型会被保存在文件系统中。随着训练效果的不断优化，可能会产生多个版本的模型，这些模型将按照既定的版本管理规则被妥善存储于文件系统中。



图 14.2.1

随后，这些模型将通过服务系统部署并上线。在上线过程中，推理系统会首先将模型加载至内存中，并对其进行版本管理，确保支持新版本的快速上线和旧版本的便捷回滚。同时，该系统还会对输入数据进行批量大小的动态优化，以适应不同的处理需求。此外，推理系统提供了多样化的服务接口（如 HTTP、gRPC 等），以便客户端轻松调用。

用户可以通过这些接口不断向推理服务系统发起请求，并接收相应的响应。除了直接面向

用户提供服务外，推理系统还可以作为一个微服务，被数据中心中的其他微服务所调用，从而在整个请求处理流程中发挥其特定的功能与职责。

14.2.2 优化目标和约束

根据图 14.2.2 所示的 AI 框架、推理系统与硬件之间的关系，可以看到，除了应对应用场景的多样化需求，推理系统还需克服由不同训练框架和推理硬件所带来的部署环境多样性挑战，这些挑战不仅增加了部署优化和维护的难度，而且容易出错。

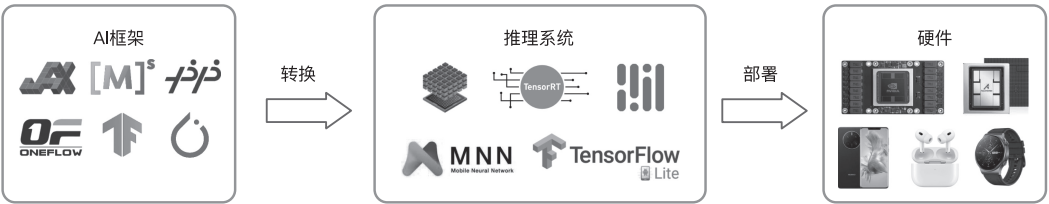


图 14.2.2

在 AI 框架方面，各种框架通常是训练而设计和优化的，因此需要考虑诸如批量大小、批处理作业对时延的不敏感性、分布式训练的支持以及更高数据精度的需求等因素。开发人员需要将各种必要的软件组件拼凑在一起，以确保兼容数据读取、客户端请求、模型推理等多个阶段的处理。此外，还需要跨越多个不断发展的框架集成和推理需求，包括对 PyTorch 等框架以及不同框架版本的支持。

在硬件方面，需要支持多种部署硬件，考虑移动端部署场景和相关约束的多样性（例如自动驾驶、智能家居等），这带来了各种空间限制和功耗约束。与此同时，大量专有场景的芯片厂商也催生了多样化的芯片体系结构和软件栈。

综上所述，可以总结设计推理系统的主要优化目标，主要有以下六点，可以通过表 14.2.1 简单了解这些目标的原因以及相关的策略。

表 14.2.1 优化目标原因与策略

优化目标	原因	相关策略
灵活性	支持多种框架，适应不同应用场景	使用模型转换工具，如 ONNX；支持多种语言接口和逻辑应用
时延	减少用户查询后的等待时间	模型压缩、剪枝、量化；优化数据预处理和后处理步骤；分布式系统设计；预测性模型加载和初始化
吞吐量	应对大量服务请求，确保服务及时性和高效性	多线程、多进程、分布式计算；服务网格；异步处理和消息队列；内存数据库和缓存
高效率	降低推理服务成本，提升系统性能	DVFS、低功耗模式；高效算法；智能调度算法
可扩展性	应对不断增长的用户或设备需求	Kubernetes 部署平台；云计算资源弹性扩展；负载均衡器
可靠性	保证推理服务稳定性和满足 SLA 要求	多服务副本和跨地域部署；故障转移机制；限流和降级策略；健康检查；数据一致性和准确性保障

1) 灵活性 (Flexibility)

灵活性是指推理系统支持多种框架的能力，为构建不同应用提供灵活性，以扩大神经网络模型部署的覆盖场景，增强部署的生产力。需要考虑 AI 框架的不断更新，特别是针对训练优化的迭代，而某些框架甚至不支持在线推理，系统需要具备足够的兼容性。

为了支持多种框架，可以利用模型转换工具，将不同框架的模型转换为一种通用的中间表示。ONNX (Open Neural Network Exchange) 是一个广泛采用的中间表示格式，它允许开发者在不同 AI 框架之间转换和部署模型。

2) 时延 (Latency)

低时延是推理系统的重要考量指标，它直接影响用户查询后获取推理结果的等待时间。推理服务通常位于关键路径上，因此预测必须快速并满足有限的尾部时延(Tail Latency)，实现次秒级别 (Sub-Second) 的时延。

在实际操作中，会运用模型压缩、剪枝、量化等技术降低模型的复杂度和大小，从而减少推理时间。

3) 吞吐量 (Throughputs)

高吞吐量是确保系统能应对大量服务请求的关键。通过提升吞吐量，系统可以服务更多的请求和用户，确保服务的及时性和高效性。

在实际操作中，可以采取多种策略来提升系统的吞吐量。首先，利用多线程、多进程或分布式计算技术，系统可以并行处理多个推理请求，显著提高处理能力。用内存数据库和缓存机制可以加快数据读写速度，减少数据访问时延，进一步提升系统性能。同时，通过减少数据在网络中的传输次数和距离，以及使用高效的数据序列化格式（如 Protocol Buffers）和压缩技术，可以降低网络时延，提高吞吐量。

4) 高效率 (Efficiency)

高效率是降低推理服务成本、提升系统性能的核心。系统应实现高效执行，低功耗使用 GPU 和 CPU，通过降本增效来优化推理服务的整体成本效益。

为了实现这一目标，可以采用动态电压频率调整 (DVFS)、低功耗模式等技术，根据实时计算需求动态调整硬件功耗。

5) 可扩展性 (Scalability)

可扩展性是应对不断增长的用户或设备需求的基础。系统需要能够灵活扩展，以应对突发和持续增长的用户请求。通过自动部署更多解决方案，随着请求负载的增加，系统能够提升推理吞吐量和可靠性。

借助底层 Kubernetes 部署平台，用户可以便捷地配置和自动部署多个推理服务副本，并通过前端负载均衡服务达到高扩展性和提升吞吐量，进一步增强推理服务的可靠性。另外，云计算平台，如 AWS、Azure、谷歌 Cloud 等提供了弹性的计算、存储和网络服务，这些服务可以根据需求快速扩展资源。

6) 可靠性 (Reliability)

可靠性是保障推理服务持续运行和用户体验的关键。系统需要具备对不一致数据、软件故障、用户配置错误以及底层执行环境故障等造成中断的弹性 (Resilient) 应对能力，以确保推理服务的稳定性和服务等级协议的达标。

为了实现高可靠性，可以部署多个服务副本，并在多个数据中心或云区域进行跨地域部署，这样即使某些组件出现故障，系统也能持续运行。在设计系统时，应考虑故障转移机制，如采用主备或多活架构，确保在发生故障时能够迅速切换到备用系统。面对高负载或潜在的故障情况，系统应通过限流和降级策略来保护自身，防止因过载而导致的系统崩溃。

14.2.3 推理系统与推理引擎

下面主要介绍推理系统与推理引擎的区别，从而更好地理解后续章节重点介绍的推理引擎的核心技术点。

1. 推理系统

如图 14.2.3 的推理系统组件与架构图所示，推理系统中常常涉及相应模块并完成相应功能，将在后面章节中逐步展开。通过图 14.2.3 可以看到推理系统的全貌。

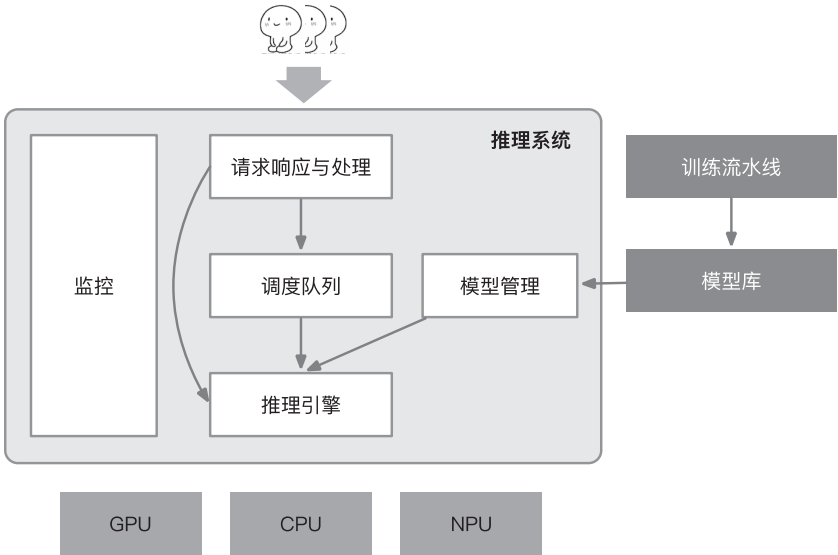


图 14.2.3

推理系统的构建涉及多个核心环节，以确保请求与响应的高效处理、资源的高效调度、推理引擎的灵活适配、模型版本的有效管理、服务的健康监控以及边缘推理芯片与代码编译的优化。

推理系统在处理请求与响应时，首要任务是高效地序列化和反序列化数据，确保后端能够迅速执行并满足严格的响应时延标准。与传统 Web 服务不同，推理系统经常需要处理图像、文本、音频等非结构化数据，这些数据的单请求或响应量通常更大。因此，推理系统需要采用高效的传输、序列化、压缩与解压缩机制，以确保数据传输的效率和性能，进而实现低时延、高吞吐量的服务。

在请求调度方面，系统可以根据后端资源的实时利用率，动态调整批处理大小、模型资源分配，从而提高资源利用率和吞吐量。当使用加速器进行推理加速时，还需考虑主存与加速器内存之间的数据传输，通过调度或预取策略，在计算的间隙中准备好数据，确保推理过程的连续性和高效性。

推理引擎是系统的核心组件，它负责将请求映射到相应的模型作为输入，并在运行时调度神经网络模型的内核进行多阶段处理。当系统部署在异构硬件或多样化的环境中时，推理引擎还可以利用编译器进行代码生成与内核算子优化，使模型能够自动转换为特定平台的高效可执行机器码，进一步提升推理性能。

在模型版本管理方面，随着云侧算法工程师不断验证和开发新版本模型，系统需要有一套完善的协议来确保版本更新与回滚的顺利进行。定期或满足一定条件的新模型会被上线替换旧模型，以提升推理服务的效果。然而，由于某些指标只能在线上测试，如果新模型效果不理想，系统还需支持回滚机制，确保模型能够迅速回退到稳定的旧版本。

健康监控是保障云侧服务稳定性的重要环节。通过可观测的日志和监控工具，服务端工程师可以实时监控服务的运行状态，及时发现并修复潜在问题，确保服务的稳定性和 SLA 达标。例如，当响应速度变慢时，运维工程师可以通过可观测的日志迅速定位瓶颈环节，并采取相应的策略进行优化，防止服务突发性无法响应。

在边缘侧等场景中，推理系统还需要面对更多样化的硬件、驱动和开发库。为了确保模型能够在这些设备上高效运行，需要通过编译器进行代码生成和性能优化，使模型能够跨设备高效执行。

2. 推理引擎

推理引擎本身可视为一种基础软件，它提供了一组丰富的 API，使得开发者能够在多种特定平台（如 CPU、GPU 和 VPU）上高效执行推理任务。以英特尔的 OpenVINO 为例，推理引擎被定义为一组 C++ 库。这些库提供了通用的 API 接口，能够在用户选择的平台上提供强大的推理解决方案。

通过推理引擎的 API，开发者可以轻松读取模型的中间表示、设置输入输出的数据格式，并在指定的设备上执行模型推理。虽然 C++ 库是主要的实现方式，但为了方便不同开发者的使用，也提供了 C 库和 Python bindings（即通过 Python 直接调用 C/C++ 库）。

图 14.2.4 是推理引擎的架构图。展示了整个推理引擎的流程结构与相关的算法，整个框架从上到下可以分为四个主要部分：API 接口输入、模型压缩与优化、Runtime 优化和 Kernel 优化。

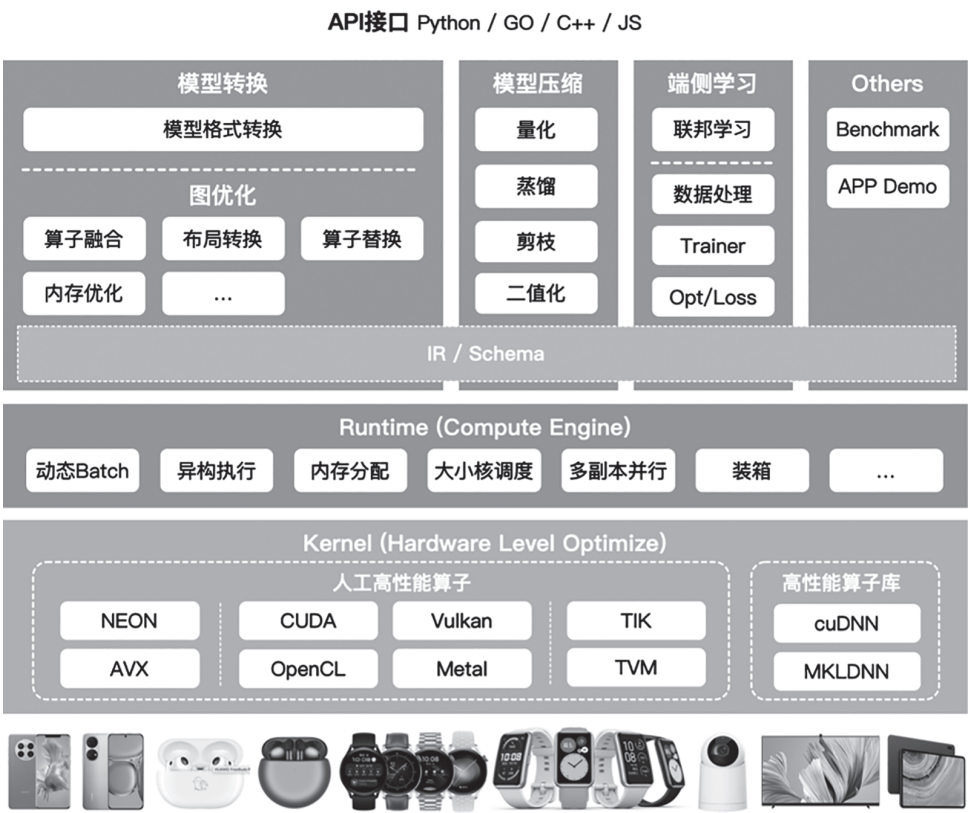


图 14.2.4

首先是 API 接口部分，这部分负责为不同编程语言（如 Python、GO、C++、JS）提供统一的接口，使得开发者能够方便地与推理引擎进行交互。这部分是推理引擎与外部环境沟通的桥梁，使得各种应用程序能够无缝地集成和使用推理引擎的功能。

接下来是模型压缩与优化部分，这个部分主要负责优化模型的大小和性能，以便模型在实际应用中更加高效地被使用。其中，包括模型格式转换、模型压缩、端侧学习、图优化等，它们能够减少模型的存储需求、加快推理速度，并提高模型的泛化能力，其中的内容将会在后续的章节中展开介绍。

IR 作为模型的标准化呈现，它在确保模型能在不同硬件和软件平台上顺畅且高效运行方面发挥着核心作用。每个具体的模型都拥有其独特的 Schema，这些 Schema 在经历必要的处理后，方能与后续的 Runtime 调度服务无缝对接，从而确保模型在各类环境中都能实现高效且稳定的运行。

在 Runtime（Compute Engine）的核心环节中，作为运行时环境或计算引擎，它专注于高效

地调度和执行模型的计算任务。在这个过程中，采用了诸如动态 Batch、异构执行和大小核调度等先进技术。动态 Batch 能够灵活适应实际需求，调整批量大小，从而优化资源利用率；异构执行支持在多类型硬件上并行计算，显著提升运算效率；大小核调度则确保在具有不同性能的核心上合理分配任务，实现整体性能的最大化。这些技术的融合应用，确保了模型在运行过程中的高效与稳定。在 Runtime 阶段，始终致力于探索如何进一步加快模型的调度和执行速度，以提供更优质的计算服务。

Kernel(Hardware Level Optimize)部分是整个流程的关键环节，它负责实际执行通过 Runtime 调度过来的模型。该部分专注于硬件级别的深度优化，利用诸如 CANN、NEON、CUDA、Vulkan 等高性能计算库，旨在显著提升推理速度。这些库针对不同硬件平台提供了精细优化后的算法和数据结构，从而最大限度地发挥硬件的性能潜力。在此环节，特别关注如何进一步提高算子的执行效率，确保整体推理性能达到最佳状态。

小结与思考

- AI 生命周期流程包括数据准备、训练与推理以及模型部署这几个组成部分。
- 推理系统是指整个 AI 模型部署和推理预测任务的执行环境。
- 推理引擎是这个系统中的核心组件，负责执行具体的模型推理任务。

14.3 推理流程全景

本节介绍神经网络模型在部署态中的两种方式：云侧部署和边缘侧部署。

其中，云侧部署适用于云服务器等具备强大计算能力和存储空间的环境，可以实现高吞吐量和集中的数据管理，但可能面临高成本、网络时延和数据隐私等挑战。

边缘侧部署适用于边缘设备和移动设备等资源受限的环境，可以通过模型优化、硬件加速和分布式计算等方式降低时延和能耗，但也面临有限算力、数据分散和安全性风险等挑战。

两种部署方式都有自己的优势和局限性，需要根据具体应用场景来选择合适的部署方式。

14.3.1 部署形态

模型的生命周期包含训练态和部署态，分别指的是 AI 模型在训练过程中的学习状态和在实际应用中的状态，它们代表了 AI 生命周期中的两个不同阶段，各自具有独特的目标、过程、环境和关注点。

部署态中的 AI 模型已经完成了训练阶段，被部署到实际应用环境中，如云服务器、边缘设备、移动应用等，用于实时或近实时地进行推理预测的过程。此时，AI 模型不再处于学习状态，而是作为服务的一部分，接受输入数据并输出预测结果。如图 14.3.1 所示，深度学习可以在边缘设备和云数据中心上执行。

云侧部署的推理系统更像传统 Web 服务，在边缘侧部署的模型更像手机应用和 IoT 应用系

统，通常指互联网上的数据中心、云服务平台或远程服务器集群，其特点是拥有强大计算能力、海量存储空间、高带宽网络连接以及丰富的管理服务。

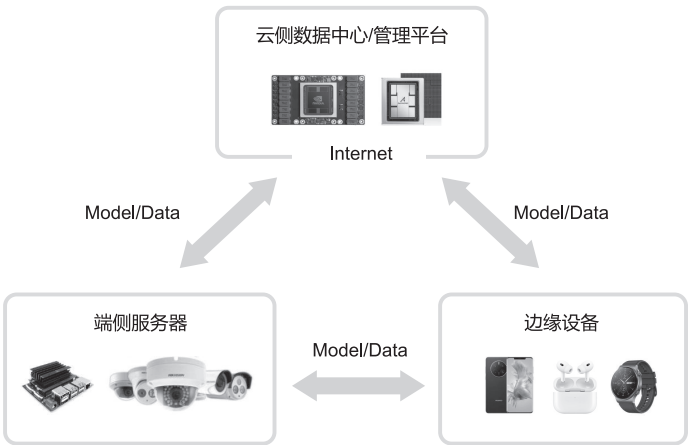


图 14.3.1

边缘侧是指靠近数据生成源或用户终端的计算设备与网络设备，其目的是减少数据传输时延、节省带宽、保护隐私或确保服务在离线或网络不稳定情况下的可用性。

部署态在云侧与边缘侧后，我们从算力、时延、网络依赖、能耗等方面对两者进行对比（表 14.3.1）。

表 14.3.1 云侧和边缘侧部署对比

	云侧部署	边缘侧部署
算力	算力强大（TFLOPS，行可扩展），适合训练和推理阶段计算	算力有限，水平扩展性差，更适合推理阶段前向计算
时延	主要的时延来自网络传输和计算开销	本地计算无网络开销或者开销很低，实时响应要求高
网络依赖	强依赖	弱依赖
能耗	百瓦以上	几十瓦，能耗比高
系统架构	开放，高度集中	封闭，架构分散
多样性	标准化程度高，CPU/GPU/NPU	多样性芯片架构，SoC 多
研发成本	配套完善，可移植性高	配套不完善，可移植性受限

14.3.2 边缘侧部署和推理方式

神经网络模型一大场景就是边缘侧部署，随着越来越多的物联网设备智能化，越来越多的移动端系统中开始部署神经网络模型。移动端部署应用常常有以下场景：智能设备、智慧城市、智能工业互联网、智慧办公室等。

边缘侧部署和推理方式常见的有 5 种（图 14.3.2）。

1. 方式一：边缘设备计算

第一种就是纯粹在边缘设备做一个推理，比如在手机、耳机、手环上面做一个简单的推理，如图 14.3.2①所示。许多研究工作都集中在如何减少深度学习在资源受限的设备上执行时的时延。

在这里，我们描述了在高效硬件和 DNN 模型设计方面的主要优化：

- 模型设计：在为资源受限的设备设计 DNN 模型时，机器学习研究人员往往侧重于设计 DNN 模型中参数量较少的模型，从而减少内存和执行时延，同时保持较高的准确性。这些模型包括 MobileNets、SqueezeNet 等。
- 模型压缩：通过模型量化、剪枝和知识蒸馏等压缩手段对模型进行压缩。
- 硬件：设计针对神经网络的专用芯片。

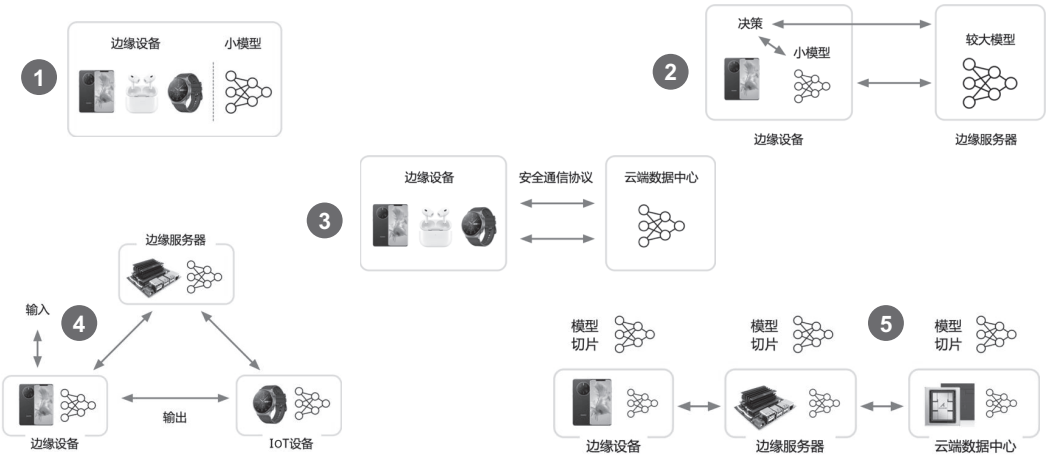


图 14.3.2

2. 方式二：安全计算

将模型部署于数据中心，边缘侧通过安全通信协议将请求发送到云侧，云侧推理返回结果，相当于将计算卸载（Offloading）到云侧。这种方式好处是利用云侧运行提升模型安全性，适合

部署端侧无法部署的大模型。完全卸载到云侧有可能违背实时性的需求。过渡方法是可以将模型切片，移动端和边缘侧各有部分模型切片。

3. 方式三：边缘设备 + 云服务器

利用深度学习的结构特点，将一部分层切分放置在设备端进行计算，其他放置在云侧，这种方式也被称作深度学习切片。这种方式一定程度上能够比方式二降低时延，由于其利用了边缘设备的算力，但是与云侧通信和计算还是会带来额外开销。这种方式的动机是，经过前几层的计算后，中间结果变得非常少。

使用这种方式部署很多时候是由于有一些数据是不出端的，用户对这些数据的隐私保护要求非常严格。例如相册的推荐，华为相册中对人像、事物、美食做了归类，这种归类数据是不出端的。这个时候会做一个小模型，在端内做一个简单的决策，这个小模型会通过大模型对大量的数据进行训练，训练完之后再推过来。

4. 方式四：分布式计算

上述方法主要考虑将计算从终端设备卸载到其他更强大的设备（如边缘服务器或云服务器）。另一种工作是从分布式计算的角度考虑问题，将 AI 任务分解并在多个辅助边缘设备上并行处理。通过精细的模型切片策略，这些边缘设备根据自身的计算和内存能力来承载不同的计算片段。在实际操作中，DeepThings 通过负载均衡来优化输入数据的处理，而 MoDNN 则采用了类似 MapReduce 的框架来管理分布式计算，提高整体效率和响应速度。

5. 方式五：跨设备卸载

最后一种方式是利用 DNN 独特的层的结构，让一些层在边缘设备上计算，一些层由边缘服务器或云侧计算。这种方法可以通过利用其他边缘设备的计算周期来潜在地提供时延减少，但是还需要注意的是，在 DNN 分区点传递中间结果的时延会影响总体的收益。

小结与思考

- 云侧部署：在云服务器上进行 AI 模型推理，具有高吞吐量和集中管理的优势，但需要解决网络时延、高成本和数据隐私保护的问题。
- 边缘侧部署：在边缘设备上执行 AI 模型推理，以降低时延和节省带宽，面临硬件资源有限、数据分散和安全性风险等挑战。

14.4 推理系统架构

推理系统架构是人工智能领域中的一个关键组成部分，它负责将训练好的模型应用于实际问题，从而实现智能决策和自动化。在构建一个高效的推理系统时，我们不仅需要考虑其性能

和准确性，还需要确保系统的可扩展性、灵活性以及对不同业务需求的适应性。

14.4.1 推理、部署、服务化

推理、部署和服务化是构建高效、可靠的机器学习系统不可或缺的三个环节。它们共同构成了从模型训练到实际应用的桥梁，确保了模型能够在真实世界中发挥其预测和决策的能力。推理阶段关注利用训练好的模型进行高效计算和生成输出，部署阶段涉及将模型迁移到生产环境并优化其性能，服务化阶段则将模型转化为可以被实际应用调用的服务。接下来，我们将探讨推理、部署和服务化这三个阶段，以及它们如何协同工作，共同支撑起一个高效、稳定的推理系统。通过对这些环节的深入理解和实践，我们可以确保机器学习模型在各种应用场景中实现最佳性能，并为用户提供可靠的解决方案。

1. 推理（Inference）

推理是指在机器学习和深度学习中使用训练好的模型对新数据进行处理并生成输出结果的过程。这个过程可以看作是模型前向计算的一部分。模型接受输入数据，通过其内部的参数和架构进行计算，最终输出预测结果。推理阶段的高效性和准确性是衡量一个模型实用价值的重要标准。

推理与预测虽然有时被混用，但在技术细节和应用场景上有所不同。推理侧重于模型内部的计算过程，而预测则更多涉及统计学和数据分析领域。在实际应用中，推理通常是模型运行的一部分，而预测可能包括更多的统计处理和数据解读。

推理过程的优化是一个重要研究方向。为了加快推理速度、减少计算资源的消耗，工程师们常常会使用模型压缩技术，如量化、剪枝和蒸馏。此外，硬件加速（如 GPU、NPU）和专用芯片（如 AI 加速器）也是提高推理效率的重要手段。

2. 部署（Deployment）

训练得到的模型并不仅仅是为了学术研究，最终目的是应用于实际问题的解决。因此，模型的部署是一个至关重要的阶段。模型部署涉及将训练好的模型从开发环境迁移到生产环境，使其能够处理真实世界的的数据并生成有用的结果。

模型部署的过程包括多个步骤和挑战：

- **移植**：将模型从开发环境迁移到生产环境，这可能涉及不同的操作系统、硬件平台和软件框架。
- **压缩**：为了在有限的计算资源下运行，模型需要进行压缩。常见的方法包括模型剪枝、量化和蒸馏，这些技术可以减少模型的参数数量和计算复杂度。
- **加速**：使用专用的硬件（如 GPU、TPU）和优化的算法（如图计算库、并行计算）来加速模型的推理过程。
- **监控和维护**：部署后的模型需要持续监控其性能，并进行必要的维护和更新，以确保其

在实际应用中始终保持高效和准确。

3. 服务化（Serving）

在模型部署的过程中，服务化是实现模型高效应用的关键步骤。服务化指的是将模型封装成一个可以供其他系统或用户调用的服务。这种服务化可以通过多种方式实现，包括 SDK 封装、应用集成和 Web 服务。

- **SDK 封装：**将模型的推理功能封装成一个软件开发工具包（SDK），开发者可以将其集成到自己的应用程序中。这种方式适用于需要在本地设备上推理的应用，如移动应用和嵌入式系统。

- **应用集成：**直接将模型集成到现有的应用程序中，使其成为应用的一部分。这种方式适用于那些需要紧密结合模型功能和业务逻辑的场景。

- **Web 服务：**将模型部署为一个 Web 服务，对外暴露 API 接口（如 HTTP(S)、gRPC 等）。这种方式使得模型可以被远程调用，适用于需要跨平台、跨设备访问模型的场景。

服务化的优势在于可以灵活地将模型功能暴露给不同的用户和系统，支持多种访问方式，并且可以方便地进行版本管理和更新。此外，通过 Web 服务，可以实现模型的分布式部署和负载均衡，从而提高系统的可用性和响应速度。

14.4.2 英伟达 Triton 推理服务

英伟达 Triton Inference Server（简称 Triton）是一个高性能、可扩展的开源推理框架。Triton 旨在为用户提供云侧和边缘侧推理的部署解决方案，支持多种神经网络模型和框架。

Triton 的主要特点包括 6 点。

- **高性能：**通过优化模型加载、执行和卸载的流程，Triton 显著提高了推理性能。
- **可扩展性：**支持水平扩展和垂直扩展，能够适应不同的计算资源和负载需求。
- **多框架支持：**兼容 TensorFlow、PyTorch、ONNX 等主流 AI 框架。
- **模型优化：**集成 TensorRT 等优化工具，进一步提升模型推理性能。
- **灵活性：**提供灵活的部署选项，支持公有云、私有云和边缘设备。
- **安全性：**支持安全传输和访问控制，保障推理服务的安全性。

作为一个强大的推理框架，Triton 能够满足多样化的 AI 应用需求，帮助企业和开发者构建高效、可靠的推理服务。Triton 还拥有活跃的开源社区，提供了丰富的文档、示例和工具，帮助开发者快速上手和部署推理服务。本小节将以 Triton 为例，介绍推理系统的架构和实现原理。

14.4.3 模型生命周期管理

模型版本管理是机器学习和深度学习中不可或缺的一部分。它允许开发者跟踪、比较和部署不同版本的模型。版本管理可能会产生不同需求，比如随着数据积累和算法改进，模型需要

不断迭代以提高性能；或是在模型开发过程中，需要记录不同实验的结果，以便比较和选择最佳模型；如果新部署的模型表现不佳，需要能够快速回滚到之前的稳定版本；而在团队中，往往需支持并行开发，即允许多个团队或个人同时在不同版本上进行开发，而不互相干扰。

金丝雀策略和回滚策略是模型生命周期管理中的典型实践，它们帮助确保模型的持续迭代和稳定性。金丝雀策略通过逐步部署新版本来降低风险，而回滚策略则提供了一种快速恢复到稳定状态的方法。

1. 金丝雀策略

金丝雀发布（Canary Deployment）是一种逐步部署和验证新版本软件的策略，得名于过去矿工用金丝雀检测矿井中是否存在有毒气体的做法。在机器学习模型或软件服务的上下文中，该策略旨在最小化新版本可能引入的风险，通过逐步将一小部分流量导向新版本，同时保持大部分流量在已知稳定的旧版本上，如图 14.4.1 所示。

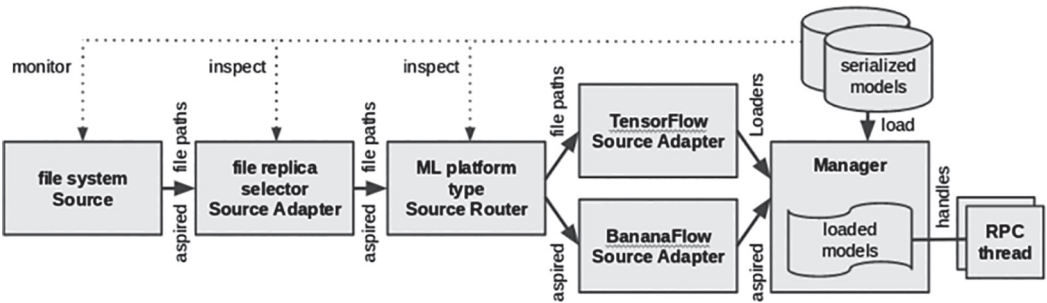


图 14.4.1

首先，需要一个灵活的路由系统，能够根据预定义的规则（如用户 ID 的哈希值、地理位置或随机分配）将请求分发到新旧两个模型版本。这样可以确保每个版本都处理一部分真实用户请求，为性能和准确性提供实际测试环境。

在推理服务过程中需要进行监控与评估，关键在于实时监控两个版本的表现，这包括但不限于精度、响应时间、资源消耗等指标。利用 A/B 测试框架，对比新旧模型在相同条件下的表现差异，确保新模型不仅在理论上改进，而且在实践中也确实提升了用户体验或业务指标。

基于监控数据，系统可以自动判断新模型是否达到预期标准，或者提供给运维人员决策依据。如果新模型表现不佳，应能快速回滚到之前的稳定版本；如果表现良好，则可逐步增加流向新模型的流量比例，直至完全替换旧版本。

● **优势：**①风险降低，通过逐步部署新模型，降低了全面部署可能带来的风险。②性能比较，允许直接比较新旧模型的性能，为决策提供数据支持。③灵活性，提供了灵活的切换机制，根据性能评估结果调整部署策略。

实施金丝雀策略需要额外的计算资源来并行运行两个模型版本，尤其是在大规模服务中。因此，合理规划资源分配，确保高峰期也能维持服务稳定性，是成功实施的关键因素之一，以避免用户暴露于可能存在缺陷的模型。

2. 回滚策略

回滚策略是故障恢复机制的重要组成部分，旨在快速应对生产环境中发现的严重问题，通过恢复到一个已知稳定的状态来最小化影响。

回滚具有快速响应机制，一旦检测到新部署的模型存在严重缺陷或性能下降，系统应立即触发回滚流程。这要求监控系统具备高度敏感性和准确性，能够迅速识别问题并触发报警。

为了实现高效的回滚，必须维护一个清晰的模型版本历史记录，并能够快速定位和部署指定的旧版本。这通常涉及版本控制系统和模型仓库的使用，确保每个模型版本的可追溯性和可部署性。

回滚过程应该尽可能平滑，减少对用户的影响。这意味着卸载有问题的新版本和装载选定的旧版本的操作顺序要经过精心设计，确保服务中断时间最短，甚至实现无中断切换。

完成回滚后，重要的是分析问题原因，修复缺陷，并通过再次实施金丝雀发布等策略安全地重新部署修复后的版本。此外，建立从回滚中得到的经验教训反馈机制，有助于优化未来的部署流程和提高系统的韧性。

● **优势：**①快速响应，允许快速从问题版本回滚到稳定版本，减少服务中断时间。②灵活性，提供了灵活的配置选项，适应不同的回滚需求。③稳定性保障，确保服务的稳定性和可靠性，保护用户免受缺陷影响。

小结与思考

- 推理系统架构是实现机器学习模型从训练到实际应用的关键桥梁，涉及推理、部署和服务化 3 个重要环节。
- 模型生命周期管理包括金丝雀策略和回滚策略，确保模型的持续迭代和稳定性，降低新版本部署风险，快速应对可能的问题。

14.5 推理引擎架构

在深入探讨推理引擎的架构之前，让我们先来概述一下推理引擎的基本概念。推理引擎作为人工智能系统中的关键组件，负责将训练好的模型部署到实际应用中，执行推理任务，从而实现智能决策和自动化处理。随着 AI 技术的快速发展，推理引擎的设计和实现面临着诸多挑战，同时也展现出独特的优势。

本节将详细阐述推理引擎的特点、技术挑战以及如何应对这些挑战，为读者提供一个较为全面的视角。同时，我们将深入探讨推理引擎的架构设计，从模型转换工具到端侧学习，再到

性能优化和算子层的高效实现，通过对这些关键技术点的一一解析，以期构建高效、可靠的推理引擎提供理论支持和实践指导。

14.5.1 推理引擎特点

推理引擎，作为人工智能和机器学习领域的重要组成部分，其设计目标在于提供一个灵活、高效且易于使用的平台，用于部署和运行已经训练好的模型，完成从数据输入到预测输出的全过程。推理引擎拥有轻量、通用、易用和高效四个特点。

- **轻量**：①资源占用少，轻量级推理引擎设计时会注重减少对计算资源（如 CPU、内存）的需求，使其能在低功耗设备上运行，如移动设备、边缘计算节点等。②体积小，引擎本身的代码库和依赖较小，便于快速部署和更新，减少对存储空间的需求。③快速启动，启动速度快，能够迅速进入工作状态，这对于需要即时响应的应用场景尤为重要。

- **通用**：①多模型支持，支持广泛的机器学习和神经网络模型格式，包括但不限于 TensorFlow、PyTorch、ONNX 等，确保了不同框架训练的模型都能被兼容和部署。②跨平台能力，推理引擎能够在多种操作系统和硬件平台上运行，无论是 Linux、Windows，还是嵌入式系统，都能保证服务的连续性和一致性。③广泛应用领域，适用于图像识别、语音处理、自然语言处理等多个领域，满足不同行业和场景的 AI 应用需求。

- **易用**：①简化部署流程，提供简洁的 API 和工具链，使得用户无需深入了解底层技术细节即可快速部署模型。②可视化工具，很多推理引擎会配套图形界面或 Web 管理面板，便于用户监控模型性能、调整参数和管理服务。③文档与社区支持，良好的文档资料和活跃的开发社区，可以帮助新用户快速上手，解决遇到的技术问题。

- **高效**：①高性能推理，通过优化算法、并行计算、硬件加速（如 GPU、TPU）等方式，最大化提升推理速度，降低时延。②资源管理，动态调整计算资源分配，根据负载自动扩缩容，确保高吞吐量的同时，也保持资源使用效率。③模型优化，支持模型压缩、量化等技术，减小模型体积，提高推理效率，尤其适合资源受限环境。

其中，易用与高效两大特性尤为关键，它们直接关系到技术解决方案的普及度与实际成效。一方面，易用性确保了技术的可接近性，加速了 AI 解决方案从概念验证到生产环境的转化过程，是推动 AI 技术从实验室走向广泛商用的桥梁。另一方面，高效性关乎推理速度和时延，还涉及资源的有效管理与优化，不仅是技术实力的体现，也是实现商业价值最大化的关键所在。

1. 轻量级

轻量级推理引擎的设计哲学围绕着“简约而不简单”的原则展开，旨在打造既功能完备又资源友好的解决方案，使之成为连接智能应用与广泛设备的桥梁。其核心优势不仅仅局限于体积小和资源占用低，更在于如何在有限的资源约束下，最大化地发挥出 AI 模型的潜力。

1) 零依赖

轻量级推理引擎从架构设计之初就追求极致的纯净与独立，确保主体功能无任何外部依赖。

这意味着，它的代码库经过精心裁剪与优化，只保留最关键、最核心的部分，从而能够轻松部署到资源受限的环境中，比如移动设备、IoT 传感器，乃至各类嵌入式系统中。这种零依赖的特性，大大简化了部署流程，降低了维护成本，使得即便是计算和存储资源有限的设备也能享受到 AI 技术带来的便利。

2) 压缩与量化

面对模型体积大、部署不便的挑战，轻量级推理引擎通过支持 FP16/INT8 精度的模型更新与量化技术，巧妙地在模型精度与体积之间找到了平衡点。FP16（半精度浮点数）相较于传统的 FP32（单精度浮点数），可以将模型大小几乎减半；而 INT8（8 位整数）量化则更为激进，通常能将模型体积压缩至原始大小的 25% ~ 50%，同时尽量保持模型的预测精度。这种压缩与量化策略，在不牺牲过多性能的基础上，大幅提升了模型部署的便捷性，让即使是复杂的神经网络模型也能轻松运行在各种轻型设备上。

2. 通用性

通用性作为推理引擎的核心特性之一，其设计目的旨在打破技术壁垒，实现无缝对接多样化需求，在模型兼容性、网络结构支持、设备与操作系统适配性上，都展现了极高的灵活性与包容性，确保了 AI 技术在广阔的应用场景中畅通无阻。

1) 广泛兼容

推理引擎的通用性首先体现在对主流模型文件格式的广泛支持上，无论是 TensorFlow、PyTorch 这类深度学习领域的重量级框架，还是 MindSpore、ONNX 这样新兴的开放标准，都能被顺利读取与执行。这意味着开发者无需担心模型来源，可以自由选择最适合的训练工具，享受技术栈的多样性。此外，对于卷积神经网络(CNN)、循环神经网络(RNN)、生成对抗网络(GAN)、Transformer 等当前主流的网络结构，引擎均能提供全面的支持，满足从图像识别、自然语言处理到复杂序列生成等多样的任务需求。

2) 动态处理

在实际应用中，模型往往需要处理不同维度、多变的数据类型。通用性推理引擎通过支持多输入多输出、任意维度的输入输出配置，以及动态输入处理能力，为复杂模型部署提供了坚实的基础。特别是对于那些含有条件分支、循环等控制流逻辑的模型，引擎同样能提供有效支持，确保这些高级功能在推理阶段得到准确执行，这对于实现更智能化、适应性更强的应用至关重要。

3) 跨平台部署

从服务器集群到个人电脑，再到手机乃至嵌入式设备，通用性推理引擎的足迹遍布所有具备 POSIX 接口的计算平台。这种跨平台的能力不仅限于硬件层面，更深入到操作系统级别。无论是 Windows、iOS、Android 这类的消费级操作系统，还是 Linux、ROS (Robot Operating System) 这类面向专业应用的操作系统，都能在其中找到推理引擎的身影。这种广泛的兼容性极大拓宽了 AI 技术的应用范围，无论是云侧大规模服务、桌面应用程序，还是移动终端乃至物联网设备，

都能轻松集成 AI 能力，释放智能潜能。

3. 易用性

易用性是衡量一个 AI 推理引擎是否能够被广泛采纳和高效利用的关键指标。它不仅要求技术方案对用户友好，还要能够降低开发门槛，提高工作效率，让开发者能够聚焦于创新，推动 AI 技术的广泛应用。

1) 算子丰富

推理引擎内置了丰富的算子库。这些算子设计用于执行常见的数值计算任务，如矩阵运算、统计分析、线性代数操作等，其功能广泛覆盖了 Numpy 这一科学计算库中的常用功能。这种设计让熟悉 Numpy 的开发者能够无缝过渡，利用熟悉的语法快速实现数据预处理和后处理逻辑，无需从头学习新的数学运算方法，大大提升了开发效率与代码的可读性。此外，通过直接映射 Numpy 接口，工程师可以轻松复用现有的 Numpy 代码片段，减少重复工作，加速项目进度。

2) 特定模块支持

推理引擎会对特定领域，如针对计算机视觉(CV)和自然语言处理(NLP)这两大核心 AI 领域，提供专门的模块与工具包，封装大量经过优化的算法与模型，使得开发者能够快速搭建起复杂的应用系统。例如，在 CV 领域，引擎一般包含图像增强、目标检测、图像分类等预置模块；而在 NLP 方面，则会提供词嵌入、语义分析、机器翻译等功能。这些模块不仅简化了模型的构建过程，还通过高度优化的实现，保障了应用的性能表现，使得开发者能够更加专注于业务逻辑的实现，而非底层技术细节。

3) 跨平台训练能力

为了满足不同场景下模型开发的需求，推理引擎不仅支持模型的跨平台推理，还扩展到了模型训练阶段。无论开发者身处何种操作系统环境，都能够便捷地进行模型的训练与微调。这种灵活性意味着研究者可以在资源丰富的服务器上训练复杂模型，随后无缝迁移到其他平台进行测试或部署，极大地促进了研发流程的连贯性和效率。同时，跨平台训练支持也意味着团队成员即便使用不同的开发环境，也能保持工作的协同性，降低了协作成本。

4) 丰富的接口与文档

强大的 API 接口是推理引擎易用性的另一大体现。良好的 API 设计应当简洁直观，同时涵盖广泛的功能，允许开发者通过少量代码就能调用复杂的内部逻辑。从模型加载、数据输入输出，到模型预测、性能监控，每一个环节都应有详尽的 API 支持。此外，高质量的文档与示例代码是不可或缺的，它们能够帮助新用户迅速上手，也为资深开发者提供深入探索的路径。通过文档和教程，用户可以快速了解如何最有效地利用引擎的各项功能，从而缩短从想法到实现的距离。

4. 高性能

高性能是推理引擎的灵魂，它直接决定了 AI 应用的响应速度、资源消耗以及用户体验。为了在多样化的硬件平台上实现最佳推理性能，推理引擎采用了多方面的策略和技术优化，确保在 iOS、Android、PC 等不同设备上，无论是哪种硬件架构或操作系统，都能充分挖掘设备潜能，实现高效运行。

1) 深度适配

针对不同设备的硬件架构和操作系统，推理引擎实现了精细的适配策略。这不仅是要支持在前几个小节中提到的“跨平台”，而是更进一步，无论是基于 ARM 还是 x86 的处理器，抑或是 iOS、Android、Windows、Linux 等操作系统，引擎都能智能识别并调整运行模式，确保在单线程下也能高效运行神经网络模型，逼近甚至达到设备的算力极限。这种深层次的适配不仅考虑了硬件的基本特性，还优化了系统调度，减少了不必要的开销，确保每一滴计算资源都被有效利用。

2) 定制优化

对于搭载 GPU、NPU 等加速芯片的设备，推理引擎进行了针对性的深度调优。例如，利用 OpenCL 框架对图形处理器进行极致的推理性能优化，确保计算密集型任务能够快速执行；而 Vulkan 方案则在减少初始化时间和优化渲染管道上展现出优势，特别适合需要快速启动和连续推理的场景。这些定制化的优化策略，确保了在各类加速硬件上的高效推理，进一步提升了整体性能表现。

3) 低级优化

为了榨干硬件最后一丝算力，推理引擎在核心运算部分采用了 SIMD 指令集和手写汇编代码。SIMD 技术允许一条指令同时对多个数据进行操作，大大提高了并行处理能力，尤其是在向量运算和矩阵乘法等常见于深度学习计算的任务中。而手写汇编则针对特定硬件指令集（如 ARMv8.2、AVX512）进行编码，通过直接控制硬件资源，实现了对特定 Kernel 算法的性能优化，这在计算密集型任务中效果显著。

4) 多精度计算

针对不同场景的性能需求，推理引擎支持 FP32、FP16、INT8 等多种精度的计算模式。这种灵活性不仅有助于在保持模型预测精度的同时，显著降低计算和内存需求，还使得引擎能够更好地匹配不同硬件对精度支持的偏好。通过采用低精度计算，推理引擎能够在一些受支持的平台上实现更快的推理速度和更优的能效比。

一款好的推理引擎可以为用户服务带来实质性的收益，如图 14.5.1 展示的柱状图，每个柱状图代表不同推理引擎在不同型号手机上的性能对比。图 14.5.1 中的推理引擎有 NCNN、MACE、TF-Lite、CoreML 和 MNN。

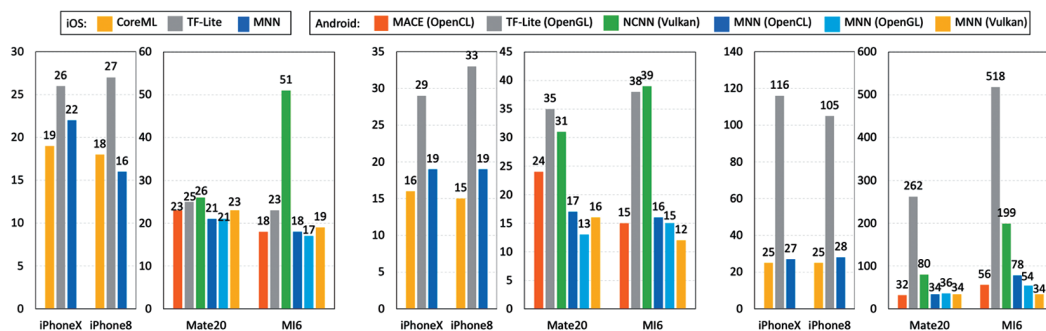


图 14.5.1

14.5.2 推理引擎技术挑战

在 AI 技术的快速发展与普及过程中，推理引擎作为连接模型与实际应用的桥梁，面临着一系列复杂的技术挑战，这些挑战主要集中在需求复杂性与程序大小的权衡、算力需求与资源碎片化的矛盾，以及执行效率与模型精度的双重要求上。

1. 需求复杂性与程序大小

随着 AI 应用领域的不断拓宽，模型的多样性和复杂度急剧增加，这给推理引擎提出了首个挑战：如何在有限的程序大小内实现对广泛模型的支持。AI 模型通常由成千上万的算子构成，涵盖了从基础的矩阵运算到复杂的卷积、递归网络等，而推理引擎必须设计出一套精简而又强大的算子集，用于模拟这些多样化的运算。这意味着引擎开发团队需要不断创新，通过算法优化、算子融合等技术，以少量的核心算子实现对各类模型的高效支持，同时还要考虑程序的可扩展性，以便未来容纳更多新型模型的接入，这无疑是一项既考验技术深度又考验策略智慧的任务。

应对 AI 推理引擎在需求复杂性与程序大小之间权衡的挑战，可以采取一系列综合策略和技术革新，确保在满足日益增长的模型复杂性需求的同时，保持程序的高效和精简。

- **模块化与插件化设计：**设计高度模块化的架构，使得每个模块专注于处理特定类型的计算或操作。这样不仅可以使核心程序保持精简，还能通过插件形式轻松添加或替换模块来支持新的模型或算子，增强系统的灵活性和可扩展性。

- **算子优化与融合：**深入分析模型中的算子，通过算法优化减少计算冗余，提高单个算子的执行效率。算子融合技术则是将多个连续且兼容的算子合并为一个，减少数据搬运和内存访问次数，从而在不增加程序大小的前提下提升运行速度。

- **动态编译与代码生成：**利用即时（JIT）编译或者静态编译时的优化技术，根据输入模型的具体结构动态生成最优化的执行代码。这种方法可以针对特定模型定制化生成执行逻辑，避

免了通用解决方案带来的额外开销，有效平衡了性能与代码体积。

2. 算力需求与资源碎片化

AI 模型的运行离不开强大的计算资源支持，尤其是面对诸如图像识别、自然语言处理等高计算量任务时。然而，实际应用场景中计算资源往往是碎片化的，从高性能服务器到低功耗的移动设备，算力和内存资源差异巨大。推理引擎需要在这片资源的“拼图”中寻找最优解，既要保证模型的高效执行，又要适应各类硬件环境。这要求引擎具备出色的适配能力，包括但不限于硬件加速技术的运用（对 GPU、NPU 加速）、动态调整算法策略以适应不同计算单元，以及智能的资源调度算法，以确保在有限的资源下也能发挥出最大的算力效能。

应对 AI 推理引擎在算力需求与资源碎片化之间的矛盾，也有一些新兴的技术方案，实现灵活应对算力需求，实现高效、低耗的模型推理，满足不同场景下的应用需求。

- **模型分层与多级缓存：**将模型拆分为不同的计算层，每层根据其计算特性和资源需求部署在最适合的硬件上。采用多级缓存策略，减少跨硬件的数据传输时延，提高整体执行效率。

- **自适应推理技术：**开发自适应推理算法，根据当前设备的硬件配置动态调整模型的精度与计算复杂度。例如，在资源有限的设备上运行轻量级模型或进行模型裁剪，而在资源丰富的环境中则加载完整模型以追求更高精度。

- **异构计算整合：**充分利用异构计算资源，通过有效的任务分解和负载均衡机制，将计算密集型任务分配给高性能硬件（如 GPU），而将控制流和轻量级计算留给 CPU，实现整体性能的最大化。

- **动态资源调度：**设计智能的资源调度算法，实时监控系统资源状态和任务队列，动态调整任务优先级和资源分配，确保高优先级或时间敏感型任务得到及时处理。

- **边缘计算与分布式推理：**利用边缘计算将部分计算任务从云侧转移到靠近数据产生的边缘设备上，减少数据传输时延。对于特别复杂的模型，可以采用分布式推理技术，将模型分割并在多台设备上并行计算，最后汇总结果。

3. 执行效率与模型精度

在追求高速推理的同时，保持模型预测的准确性是另一个核心挑战。一方面，为了提高执行效率，模型压缩、量化技术常被采用，这虽然能显著减少模型体积、加快推理速度，但可能会以牺牲部分精度为代价。另一方面，业务场景往往对模型的精度有着严格要求，特别是在医疗诊断、金融风险评估等领域，任何微小的精度损失都可能导致严重后果。因此，推理引擎在设计上必须巧妙平衡这两方面的需求，通过算法优化、混合精度推理，以及对特定模型结构的定制优化等策略，力求在不明显影响模型精度的前提下，实现推理效率的最大化。这不仅是一个技术挑战，也是对工程实践智慧的考验，需要不断地试错、优化与迭代，以找到最合适的平衡点。

应对 AI 推理引擎在执行效率与模型精度的双重要求，则可以使用一些关键策略，满足多样

化业务场景的严苛要求。

- **知识蒸馏**: 通过知识蒸馏技术, 用一个大而精确的教师模型去训练一个较小的学生模型, 让学生模型在保持较高精度的同时拥有更快的推理速度。这种方法可以在不直接牺牲模型精度的前提下, 实现模型的小型化和效率提升。

- **量化与微调**: 虽然量化技术会降低模型的精度, 但通过细致的量化方案选择 (如对敏感层采取不同的量化策略) 和后续的微调步骤, 可以在很大程度上恢复损失的精度。微调过程可以让模型在量化后的新精度水平上重新学习, 优化权重, 最小化精度损失。

- **模型剪枝与稀疏化**: 通过对模型进行剪枝, 移除对最终预测贡献较小的权重或神经元, 减少计算量。同时, 利用稀疏计算技术进一步减少计算负担, 许多现代硬件已支持高效的稀疏矩阵运算。精心设计的剪枝策略可以大幅降低计算需求而不显著影响模型表现。

- **缓存与预计算策略**: 对频繁访问的数据或计算结果进行缓存, 减少重复计算, 特别是在循环或递归模型中。预计算某些静态或几乎不变的特征, 进一步加速推理流程。

14.5.3 整体架构

推理引擎作为人工智能应用的核心组件, 其架构设计直接关系到模型在实际部署环境中的效率、灵活性和资源利用率。整体架构可以细分为优化阶段和运行阶段, 每个阶段都包含了一系列关键技术以确保模型能够高效、准确地运行于目标设备上。

优化阶段聚焦于将训练好的模型转换并优化成适合部署的形式:

- **模型转换工具**负责将模型从研究阶段的格式转换为高效执行的格式, 并进行图优化, 减少计算负担。

- **模型压缩**通过技术如剪枝、量化、知识蒸馏等减小模型大小, 使之更适用于资源有限的环境。

- **端侧学习**允许模型在部署后继续学习和适应新数据, 无需返回服务器重新训练, 提升了模型在特定场景或用户个性化需求下的表现。

- **其他组件**包括 Benchmark 工具, 用于性能评测和调优指导, 以及应用演示 (App Demos), 服务于模型能力展示与实战反馈收集, 共同助力模型的高效部署与持续优化。

运行阶段确保模型在目标设备上的高效执行:

- **调度层**管理模型加载、资源分配及任务调度, 根据设备情况灵活安排计算任务。

- **执行层**直接执行模型计算, 针对不同硬件优化运算逻辑, 有效利用 CPU、GPU 等资源。

1. 模型转换工具

模型转换工具是 AI 应用部署流程中的基石, 它不仅涉及将模型从训练环境迁移到推理环境的基本格式转换, 还深入计算图级别的精细优化, 以确保模型在目标平台上高效、稳定地运行 (图 14.2.4)。

1) 模型格式转换

模型转换首先面临的是格式的跨越。想象一个模型，最初在如 TensorFlow、PyTorch 或 MindSpore 这样的科研友好型框架下被训练出来，它的原始形态并不直接适用于生产环境中的推理引擎。因此，转换工具承担起了“翻译者”的角色，将模型从其诞生的框架语言翻译成一种或多种行业广泛接受的标准格式，如 ONNX，这不仅增强了模型的可移植性，也为模型的后续处理和部署提供了通用的接口。

- **跨框架兼容性：**支持将模型从一种框架（如 TensorFlow、PyTorch）转换为另一种（如 ONNX、TensorRT），使得模型能够在不同的推理引擎上执行，增强了应用开发的灵活性和平台的通用性。

- **版本适应性：**解决因框架升级导致的模型兼容问题，确保旧模型可以在新版推理引擎上正确运行，或新模型能回溯支持老版本系统。

- **标准化输出：**转换后的模型通常被格式化为一种或多种行业标准格式，如 ONNX，这种标准化促进了生态系统中工具和服务的互操作性。

2) 计算图优化

然而，格式转换仅仅是冰山一角。真正的挑战在于如何对计算图进行深度优化，这是决定模型能否高效执行的关键。计算图，作为神经网络结构的数学抽象，其优化涉及对图结构的精细剖析与重塑。其中，算子融合、布局转换、算子替换与内存优化构成了优化的核心四部曲，每一步都是对模型性能极限的深入探索与精心雕琢。

● 算子融合

算子融合宛若匠人之手，将计算图中相邻且兼容的多个基本运算合并为一个复合操作。这一过程减少了运算间的数据传输成本，消除了不必要的内存读写，使得数据流动更为顺畅。例如，将 Conv 卷积操作紧随其后的 BatchNorm 归一化和 ReLU 激活函数融合为一体（参见第 10 章图 10.3.7），不仅缩减了计算图的复杂度，还充分利用了现代硬件对连续计算的支持，加速了计算流程。融合策略的巧妙应用，要求对底层硬件架构有着深刻理解，确保每一步融合都精准贴合硬件的并行计算优势。

● 布局转换

布局即数据在内存中的组织方式。布局转换优化是对数据存取路径的重构，旨在最小化内存访问时延，最大化数据复用，特别是在神经网络模型广泛依赖的矩阵运算中，合理的布局选择能显著提升计算密集型任务的执行速度。例如，从 NHWC 到 NCHW 的转换，或是其他特定硬件偏好的布局格式调整，虽看似简单，实则深刻影响着内存访问模式与带宽利用。

如 NHWC 是 TensorFlow 等一些框架中默认使用的布局，尤其在 CPU 上较为常见。N 代表批量大小 (Batch Size)，H 代表高度 (Height)，W 代表宽度 (Width)，C 代表通道数 (Channels)，RGB 中，C=3。NCHW 布局更受 CUDA 和 cuDNN 等 GPU 库的青睐，尤其是在进行深度学习加速时。

例如，一个处理单张 RGB 图像的张量布局为 NHWC 时，形状表示为 [1, 224, 224, 3]，意味

着 1 个样本，图像尺寸为 224 像素×224 像素，3 个颜色通道。同样的例子，在 NCHW 布局下，张量形状会是[1, 3, 224, 224]。假设我们要在一个简单的 CNN 层中应用卷积核，对于每个输出位置，都需要对输入图像的所有通道执行卷积计算。

NHWC 布局下，每次卷积操作需要从内存中顺序读取不同通道的数据（因为通道数据交错存储），导致频繁的内存访问和较低的缓存命中率。

NCHW 布局下，由于同一通道的数据连续存储，GPU 可以一次性高效地加载所有通道的数据到高速缓存中，减少了内存访问次数，提升了计算效率。

因此，虽然布局转换本身是一个数据重排的过程，但它能够显著改善内存访问模式，减少内存带宽瓶颈，最终加速模型的训练或推理过程，特别是在硬件（如 GPU）对特定布局有优化的情况下。

● 算子替换

面对多样化的硬件平台，原生模型中的某些算子可能并非最优选择。算子替换技术正是基于此洞察而生，通过对计算图的深度剖析，识别那些性能瓶颈或不兼容的算子，并以硬件友好、效率更高的等效算法予以替代。这一替换策略，如同为模型量身定制的高性能组件升级，不仅解决了兼容性问题，更是在不牺牲模型准确性的前提下，挖掘出了硬件潜能的深层价值。

● 内存优化

在计算资源有限的环境下，内存优化是决定模型能否高效运行的关键。这包括但不限于通过循环展开减少临时变量，采用张量复用策略减小内存占用，以及智能地实施缓存策略加速重复数据的访问。内存优化是一场对空间与时间的精密权衡，它确保模型在推理过程中既能迅速响应，又能保持较低的内存足迹，尤其在嵌入式系统或边缘设备上，这一优化的重要性尤为凸显。

2. 模型压缩

模型压缩作为 AI 领域的一项核心技术，也是推理引擎架构中不可缺少的一部分，它旨在通过一系列精巧的策略减少模型大小，同时保持其预测性能尽可能不变，甚至在某些情况下加速训练和推理过程。这一目标的实现，离不开量化、知识蒸馏、剪枝以及二值化等关键技术的综合运用，它们各自以独特的方式对模型进行“瘦身”，而又尽可能不牺牲其表现力。这些内容将会在后续章节中详细讲解，故此处只作简单介绍。

1) 量化

量化技术的核心思想在于，将模型中的权重和激活函数从高精度浮点数转换为低精度数据类型，如 8 位整数甚至二进制形式。这一转换不仅显著降低了模型的存储需求，也因为低精度运算在现代硬件上的高效实现而加速了推理过程。当然量化过程中需要精心设计量化方案，如选择合适的量化区间、量化策略和误差补偿方法，以确保精度损失控制在可接受范围内，实现性能与精度的平衡。

2) 知识蒸馏

知识蒸馏，一个形象的比喻，是指利用一个庞大而复杂的“教师”模型（通常是准确率较高的模型）来指导一个较小的“学生”模型学习，使其在保持相对较高精度的同时，模型规模大幅减小。这一过程通过让学生模型模仿教师模型的输出分布或者直接利用教师模型的软标签进行训练，实现了知识的传递。知识蒸馏不仅限于模型大小的缩小，还为模型的轻量化设计开辟了新的思路，尤其是在资源受限的设备上部署模型时。

3) 剪枝

剪枝技术，正如其名，旨在去除模型中对预测贡献较小或冗余的权重和连接，实现模型结构的简化。这包括但不限于权重剪枝、通道剪枝和结构化剪枝等策略。通过设定一定的剪枝阈值或利用稀疏性约束，模型中的“无用枝条”被逐一识别并移除，留下的是更为精炼的核心结构。值得注意的是，剪枝过程往往伴随有重新训练或微调步骤，以恢复因剪枝可能带来的精度损失，确保模型性能不受影响。

4) 二值化

二值化，顾名思义，是将模型中的权重乃至激活值限制为仅有的两个离散值（通常是+1和-1）。这种极端的量化方式进一步压缩了模型体积，简化了计算复杂度，因为二值化运算可以在位级别高效实现。尽管二值化模型在理论上极具吸引力，实践中却面临着精度下降的挑战，需要通过精心设计的训练策略和高级优化技术来弥补。

3. 端侧学习

端侧学习，作为人工智能领域的一个前沿分支，致力于克服传统云中心化模型训练的局限，通过将学习能力直接赋予边缘设备，如手机、物联网传感器等，实现数据处理的本地化和即时性（图 14.2.4）。这一范式的两大核心概念——增量学习和联邦学习，正在重新定义 AI 模型的训练和应用方式，为解决数据隐私、网络时延和计算资源分配等问题提供了创新途径。

为了支撑高效的端侧学习，一个完备的推理引擎不仅仅是模型执行的平台，它还需要集成数据处理、模型训练（Trainer）、优化器（Opt）以及损失函数（Loss）等核心模块，形成一个闭环的端到端解决方案。

1) 数据处理模块

在端侧学习场景下，数据处理模块需要特别考虑资源限制和隐私保护。它负责对原始数据进行清洗、转换和标准化，确保数据格式符合模型输入要求。考虑端设备的计算和存储限制，此模块还应实现高效的数据压缩和缓存策略，减少内存占用和 I/O 操作。例如，采用差分编码或量化技术减少数据传输量，并利用局部数据增强技术提高模型泛化能力，而无需频繁访问云侧数据。

2) Trainer 模块

Trainer 模块在端侧学习中扮演着模型更新与优化的关键角色。不同于云侧的大规模训练，端侧训练往往侧重于模型的微调或增量学习。此模块需要实现轻量级的训练循环，支持快速迭

代和低功耗运行。它通过与优化器模块紧密集成，根据从数据处理模块接收到的数据，逐步调整模型权重。在资源受限环境下，Trainer 模块还需支持断点续训和模型检查点保存，确保训练过程的连续性和可靠性。

3) 优化器 (Opt) 模块

优化器模块选择和实施合适的算法来最小化损失函数，指导模型权重的更新。在端侧学习中，常用的优化器如 Adam、RMSProp 等，需要进行定制优化，以减少内存使用和计算复杂度。例如，采用稀疏梯度优化或低精度计算（如 16 位浮点数）来加速训练过程，同时保持模型性能。

4) 损失函数 (Loss) 模块

损失函数定义了模型学习的目标，直接影响模型的预测能力和泛化能力。在端侧学习场景中，损失函数的设计不仅要考虑标准的分类或回归任务需求，还要能反映特定的业务目标或约束条件，比如模型的大小、推理速度或隐私保护需求。例如，可能会采用带有正则化项的损失函数，以促进模型的稀疏性，减少模型尺寸，或者设计隐私保护相关的损失函数，确保模型学习过程中数据的隐私安全。

5) 增量学习

增量学习，顾名思义，是一种让模型在部署后继续学习新数据、适应新环境的能力。不同于一次性大规模训练后便固定不变的传统模型，增量学习模型能够根据设备端接收到的新信息逐步自我更新，实现持续的性能优化。这一过程类似于人类的渐进式学习，模型在不断接触新案例的过程中，逐渐积累知识，优化决策边界。技术上，增量学习需克服遗忘旧知识（灾难性遗忘）的问题，通过算法如学习率调整、正则化策略、经验回放等手段保持模型的泛化能力，确保新旧知识的和谐共存。

个性化推荐系统是增量学习的一个典型应用领域。在新闻、音乐或购物应用中，用户每次点击、评分或购买行为都能被模型捕捉并即时反馈至模型，通过增量学习调整推荐算法，使得推荐结果随着时间推移更加贴合用户的个性化偏好。例如，Spotify 的 Discover Weekly 功能，就能通过持续学习用户的听歌习惯，每周生成个性化的播放列表，展现了增量学习在提升用户体验方面的巨大潜力。

6) 联邦学习

联邦学习，则为解决数据隐私和跨设备模型训练提供了一条创新路径。在这一框架下，用户的个人数据无需上传至云侧，而是在本地设备上对模型参数进行更新，之后仅将这些更新（而非原始数据）分享至中心服务器进行聚合，形成全局模型。这一过程反复进行，直至模型收敛。联邦学习不仅保护了用户隐私，减少了数据传输的负担和风险，还允许模型从分布式数据中学习更加丰富和多样化的特征，提升了模型的普遍适用性。其技术挑战在于设计高效且安全的参数聚合算法，以及处理设备异构性和通信不稳定性带来的问题。

(1) 横向联邦学习聚焦于那些拥有相同特征空间（即模型输入维度相同）但样本空间不同（覆盖不同用户或数据实例）的参与方。想象一个跨国企业，其在全球不同地区设有分支，每个分支收集了当地用户的购买数据，尽管数据包含的属性（如年龄、性别、购买历史）一致，但

记录的顾客群体各异。在横向联邦学习中，这些分支机构无需交换各自的具体用户数据，而是各自利用本地数据训练模型，仅分享模型参数的更新（如梯度或权重变化）到中央服务器。服务器汇总这些更新，更新全局模型后，再分发回各个分支。如此循环，直至模型收敛。这种方式适用于用户特征重叠度高而用户覆盖范围广的场景，如多银行间联合欺诈行为的检测。

（2）与横向联邦学习相反，纵向联邦学习适用于那些数据集中包含大量重叠用户（样本空间相同）但特征维度不同（即各参与方掌握的用户属性不同）的场景。以银行与电商平台的合作为例，银行掌握用户的财务信息（如信用记录、收入水平），而电商平台则拥有消费者的购物行为数据（浏览历史、购买偏好）。两者虽然覆盖的用户群体可能高度重叠，但所拥有的数据特征却互为补充。在纵向联邦学习中，通过在服务器端设计特殊协议，使得不同特征的数据能够在不直接交换的前提下，协同参与模型训练。这可能涉及特征对齐、安全多方计算等技术，以确保特征的隐私和安全。通过这种合作，银行和电商可以共同构建一个更全面的用户画像模型，用于个性化推荐或风险评估，而无需泄露各自的敏感数据。

4. 中间表示

在现代推理引擎的设计与实现中，中间表示扮演了至关重要的角色，它是连接模型训练与实际推理执行之间的桥梁。中间表示的核心目标是提供一种统一、高效的模型描述方式，使得不同来源、不同架构的模型能够被标准化处理，进而优化执行效率并增强平台间的兼容性。这一概念深入到模型优化、编译及执行的每一个环节，其重要性不言而喻（图 14.2.4）。

中间表示为模型提供了丰富的优化空间。对计算图的优化工作大都集中在对模型进行中间表示之后，通过静态分析、图优化等技术，可以对模型进行裁剪、融合、量化等操作，减少计算量和内存占用，提升推理速度。这一过程如同将高级编程语言编译为机器码，但面向的是神经网络模型。

统一的中间表示形式确保模型能够在云、边、端等多类型硬件上自由部署，实现一次转换、处处运行的目标。它简化了针对特定硬件的适配工作，使得模型能在不同环境间无缝迁移，满足多样化应用需求。

围绕中间表示，可以形成一个包含工具链、库函数、社区支持在内的完整生态系统。开发者可以利用这些资源快速实现模型的调试、性能监控和持续优化，加速产品从原型到生产的整个周期。

1) Schema

Schema 作为中间表示的一部分，定义了一套规则或者说是结构化框架，用于描述模型的组成要素及其相互关系。它类似于一种“词汇表”和“语法规则”，使得模型的每一层、每个操作都被赋予了明确且规范的意义。通过 Schema，复杂的神经网络结构可以被抽象为一系列基本操作单元的组合，如卷积、池化、全连接层等，这不仅简化了模型的表示，也为后续的优化提供了基础。

2) 统一表达

“统一表达”的理念在于打破模型表述的壁垒，无论原始模型是基于 TensorFlow、PyTorch、

MXNet，还是其他任何框架构建，一旦转换为中间表示形式，它们都将遵循一套共同的语言体系。这种统一性极大降低了模型迁移的成本，使得开发者无需担心底层实现细节，就能在不同的推理引擎或硬件平台上复用模型。更重要的是，它促进了模型优化技术的共享与迭代，因为优化算法可以直接作用于这种标准表示之上，而无需针对每种框架单独开发适配器。

5. Runtime

Runtime，即推理引擎的执行引擎，负责将中间表示形式的模型转换为可执行的指令序列，并将其部署到目标设备上执行（图 14.2.4）。执行引擎不仅仅涉及模型的加载与执行两个基本步骤，还深入涵盖了多种策略和技术，以优化资源利用、提升运行效率，确保在多样化的硬件平台上都能实现高性能表现。我们以自动驾驶为例，来介绍 **Runtime** 技术在模型推理中的作用。

1) 动态 Batch 处理

动态批（Batch）处理技术为推理引擎带来了前所未有的灵活性，它允许系统根据实时的系统负载状况动态地调整批量大小。在负载较轻的时段，如清晨或深夜，当车辆较少、系统接收的图像帧数量降低时，推理引擎能够智能地将多个图像帧合并成一个较大的批次进行处理。这一策略不仅显著提高了硬件资源的利用率，如 GPU 的大规模并行处理能力，而且减少了单位请求的计算开销，使系统能够在较低的负载下维持高效的推理性能。

反之，在高峰时段或紧急情况下，当系统面临高负载的挑战时，动态批处理技术能够迅速减少批量大小，确保每个请求都能得到及时响应，从而保证了自动驾驶系统的即时性和安全性。这种能够根据实时负载动态调整批量大小的能力，对于自动驾驶系统应对不可预测的流量波动至关重要，它不仅提升了系统的稳定性，还确保了在不同负载情况下系统都能保持高效运行。

2) 异构执行

现代硬件平台融合了多元化的计算单元，包括 CPU、GPU 以及 NPU 等，每种处理器都拥有其独特的优势。异构执行策略通过智能分配计算任务，能够充分利用这些不同处理器的性能特点。具体而言，该策略会根据模型的不同部分特性和当前硬件状态，将计算任务分配给最合适的处理器执行。例如，对于计算密集型的卷积操作，它们通常会被卸载到 GPU 或 NPU 上执行，因为这类处理器在处理大量矩阵运算时表现出色；而涉及复杂控制流和数据预处理的任務，则更适合交由 CPU 来处理。

对于自动驾驶，物体检测模型经常需要执行多种类型的计算任务。其中，卷积层由于其计算密集型的特性，非常适合在 GPU 或 NPU 上执行，以充分利用其强大的并行处理能力。而逻辑判断、数据筛选等依赖复杂控制流的操作，则更适合在 CPU 上执行。通过采用异构执行策略，自动驾驶系统的推理引擎能够自动将卷积层任务调度到 GPU 等高性能处理器上，同时将数据预处理和后处理任务分配给 CPU 处理，从而实现整体计算流程的高效与快速。这种策略不仅提升了系统的性能，还确保了自动驾驶系统在各种场景下都能保持出色的响应速度和准确性。

3) 内存管理与分配

在推理过程中，高效的内存管理和分配策略是确保运行效率的重中之重。这些策略涵盖

了多个方面，如重用内存缓冲区以减少不必要的复制，智能地预加载模型的部分数据到高速缓存中以降低访问时延，以及实施内存碎片整理机制来最大化可用内存资源。这些措施不仅有助于降低内存占用，还能显著提升数据读写速度，对模型的快速执行起到至关重要的作用。

在自动驾驶车辆的过程中，实时处理高分辨率图像对内存资源提出了极高的挑战。为了应对这一挑战，推理引擎一般采用智能化的内存管理策略。例如，通过循环缓冲区重用技术，推理引擎能够在处理新图像帧之前，复用先前帧所占用的内存空间来存储特征图等中间结果。这种做法不仅避免了频繁的内存分配与释放操作，减少了内存碎片的产生，还显著地提高了内存使用效率和系统的整体响应速度。这些精细化的内存管理策略确保了自动驾驶系统能够高效、稳定地运行，为实时、准确的决策提供支持。

4) 大小核调度

在移动设备上，大核（高性能核心）与小核（低功耗核心）之间的性能差异显著，为了最大化硬件资源的使用效率，推理引擎必须能够动态调整计算任务在这两种核心之间的分配。这要求推理引擎具备先进的调度能力，以便根据当前的任务负载和类型，智能地将任务分配给最合适的处理器核心。

对于采用大小核的处理器，推理引擎可以采用精细化的任务调度策略。具体来说，它可以将计算密集型、对性能要求高的任务，如自动驾驶中的车辆路径规划等复杂逻辑运算，分配给高性能大核处理，以确保足够的计算能力和处理速度。而对于数据预处理、简单状态监控等轻量级任务，则可以交给低功耗小核完成，以节约能耗并延长续航时间。

在自动驾驶的场景下，推理引擎的这种动态负载均衡能力显得尤为重要。它可以根据车辆实际运行中的任务需求，实时调整任务在大小核之间的分配，从而在确保处理复杂任务时拥有足够算力的同时，也能在执行轻量级任务时有效降低能耗，为自动驾驶系统提供更为高效、稳定和节能的运行环境。

5) 多副本并行与装箱技术

在分布式系统或多核处理器架构中，多副本并行技术展现出其独特的优势。该技术通过创建模型的多个副本，并分配给不同的计算单元进行并行执行，实现了线性或接近线性的性能加速。与此同时，装箱（Batching）技术作为一种有效的并行处理策略，通过将多个独立的请求合并成一个批次进行集中处理，显著提升了系统在面对大量小型请求时的吞吐量和资源利用率。

在自动驾驶的实时应用中，对时延响应的要求极为严格。多副本并行技术在这里得到了完美的应用，特别是在多 NPU 系统中。一方面，每个 NPU 运行模型的一个副本，能够同时处理多个传感器数据流，实现并行推理，从而大幅缩短决策时间，确保车辆能够快速响应各种路况。另一方面，装箱技术在处理单个高分辨率图像帧时同样表现出色。通过将图像分割成多个小区域，每个区域作为一个小批次进行处理，不仅保证了实时性，还显著地提高了系统的整体吞吐量，使自动驾驶系统能够在复杂的驾驶环境中保持高效运行。

6. 高性能算子

高性能算子层是推理引擎架构中的关键组成部分，它主要负责对模型中的数学运算进行优化、执行和调度（图 14.2.4）。该层的主要任务是将模型中的计算任务分解为一系列基础算子，然后通过各种方法对这些算子进行优化，以提高模型的运行效率。

1) 算子优化

算子优化是提高模型运行效率的关键。通过对模型中的算子进行优化，可以有效地减少计算量、降低内存使用、提高计算速度。常见的优化方法包括：

- **融合优化**：在神经网络模型中，许多算子之间可能存在冗余的计算步骤或内存拷贝操作。融合优化旨在将相邻的算子合并成一个单一算子，从而减少整体的计算次数和内存使用。这种优化策略能够显著提高模型的计算效率，特别是在硬件加速器（如 NPU）上运行时效果更为显著。

- **量化优化**：量化是一种将浮点数运算转换为整数运算的技术，以减少计算复杂性和提高模型运行速度。通过降低数据的表示精度，量化可以减少计算所需的资源，并可能使模型在资源受限的设备上运行。尽管量化可能会导致模型精度的轻微下降，但在许多应用中，这种精度损失是可以接受的。

- **稀疏优化**：稀疏优化是针对稀疏矩阵或稀疏张量进行的特殊优化。在神经网络模型中，权重矩阵或特征张量可能包含大量的零值元素，这些零值元素在计算过程中不会产生任何贡献。通过稀疏优化，我们可以跳过这些无效计算，只处理非零元素，从而提高计算效率。例如，使用稀疏矩阵乘法算法来替代常规的矩阵乘法算法，可以显著减少计算量。此外，稀疏优化还可以与量化优化相结合，进一步降低计算复杂度和内存占用。

2) 算子执行

算子执行是指将优化后的算子在硬件上执行的过程。不同的硬件平台有不同的执行方式。

- **CPU 执行**。在 CPU 上执行算子时，通常会利用 CPU 的 SIMD 指令集来加速计算。SIMD 指令集允许 CPU 同时处理多个数据元素，通过并行执行相同的操作来显著提高计算速度。常见的 SIMD 指令集包括 Intel 的 AVX 和 ARM 的 NEON 等。这些指令集可以极大地提升矩阵运算、图像处理等任务的执行效率。通过优化算子以充分利用这些指令集，我们可以在 CPU 上实现更快的计算速度。

- **GPU 执行**。在 GPU 上执行算子时，通常会使用 CUDA、OpenCL 或 Metal 等接口来编写并行计算代码。这些接口提供了丰富的并行计算功能和高效的内存访问机制，使得开发者能够轻松地在 GPU 上实现大规模并行计算。通过利用 GPU 的并行计算能力，我们可以显著加速神经网络模型的训练和推理过程。

与 CPU 相比，GPU 在并行计算方面具有明显的优势。GPU 拥有更多的计算核心和更高的内存带宽，能够同时处理更多的数据。此外，GPU 还支持更高级别的并行性，如线程级并行和指令级并行等。这使得 GPU 在执行深度学习算子时，能够更充分地利用硬件资源，实现更高的

计算效率。

3) 算子调度

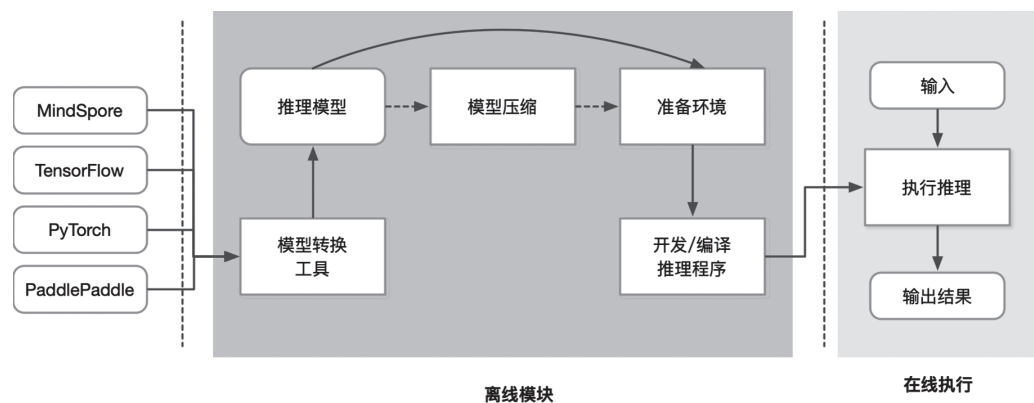
算子调度是高性能计算中至关重要的一个环节，它涉及根据硬件资源的实际可用性和算子的特性，合理规划和决定算子的执行顺序以及执行位置。在构建高性能算子层时，算子调度策略的选择直接影响整个系统的计算效率和性能。

● **异构调度。**异构调度是一种智能的算子调度策略，它根据算子的类型、复杂度和计算需求，结合不同硬件的特性（如 CPU、GPU、FPGA 等），将算子分配到最适合其执行的硬件上。例如，对于计算密集型算子，可能会优先将其调度到 GPU 上执行，因为 GPU 拥有更多的计算核心和更高的并行计算能力；而对于需要频繁内存访问的算子，可能会选择在 CPU 上执行，因为 CPU 在内存访问方面更具优势。通过异构调度，可以充分发挥不同硬件的优势，实现计算资源的最大化利用，从而提高整体的计算效率。

● **流水线调度。**流水线调度是一种高效的算子调度策略，它将多个算子按照一定的逻辑顺序排列，形成一条计算流水线。在流水线上，每个算子都有自己的处理单元和缓冲区，可以独立地进行计算，而无需等待前一个算子完成全部计算。当第一个算子完成一部分计算并将结果传递给下一个算子时，第一个算子可以继续处理新的数据，从而实现连续的、并发的计算。流水线调度可以极大地提高计算速度，减少等待时间，使得整个系统的吞吐量得到显著提升。

14.5.4 推理流程

根据以上的推理引擎结构介绍，我们可以得出以下的推理流程，这个流程涉及多个步骤和组件，包括其在离线模块中的准备工作和在线执行的过程，它们共同协作以完成推理任务（图 14.5.2）。



首先，推理引擎需要处理来自不同 AI 框架的模型，比如 MindSpore、TensorFlow、PyTorch 或者 PaddlePaddle。这些框架训练得到的模型将被送至模型转换工具，进行格式转换，以适配推理引擎的特定格式。

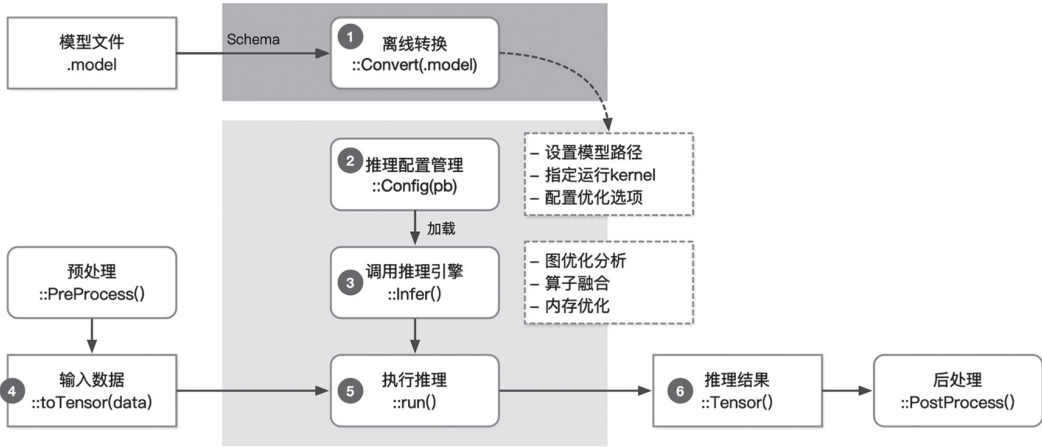
转换后得到的推理模型，需要进行压缩处理。模型压缩是推理引擎中常见的步骤，因为未压缩的模型在实际应用中很少见。模型压缩后需要准备环境，这一步骤涉及大量配置工作，包括大小核的调度、模型文档的获取等，确保模型能够在正确的环境中运行。

完成环境准备后，推理引擎会进行开发和编译，生成用于执行推理的进程。这个推理进程是实际执行推理任务的核心组件，它依赖于推理引擎提供的 API，为用户提供模块或任务开发所需的接口。

开发工程师会按照这个流程进行工作，开发完成后，推理引擎将执行推理任务，使其在运行时(Runtime)工作。此时，推理引擎的执行依赖于输入和输出的结果，这涉及在线执行的部分。

14.5.5 开发推理程序

开发推理程序是一个复杂的过程，涉及模型的加载、配置、数据预处理、执行推理以及结果的后处理等多个步骤。图 14.5.3 仅简单提供一个示例，介绍如何开发一个推理程序。



1) 模型转换

首先，需要将训练得到的模型转换为推理引擎能够理解的格式。这一步骤通常由模型转换工具完成，形成一个统一表达的格式。

2) 推理配置管理

这通常涉及设置模型路径、选择运行的设备（如 CPU、GPU 等），以及是否开启计算图优化等。一个 AI 框架会提供相关的 API 来设置这些选项，并在模型加载时自动应用。

```
Config config;
config.setModelPath("path_to_model");
config.setDeviceType("GPU");
config.setOptimizeGraph(true);
config.setFusion(true); // 开启算子融合
config.setMemoryOptimization(true); // 开启内存优化
```

3) 调用推理引擎

一旦配置选项设置完毕，下一步就是创建推理引擎对象。这个对象将负责管理整个推理过程，包括加载模型、执行推理等。创建推理引擎对象通常需要传递配置对象作为参数。

```
Predictor predictor(config);
```

4) 准备输入数据

在执行推理之前，必须准备好输入数据。首先，这包括对原始数据进行预处理，比如减去均值、缩放等，以满足模型的输入要求。然后，需要获取模型所有输入 Tensor 的名称，并通过推理引擎获取每个输入 Tensor 的指针，最后将预处理后的数据复制到这些 Tensor 中。

```
// 预处理数据
auto preprocessed_data = preprocess(raw_data);

// 获取输入Tensor名称和指针
auto input_names = predictor->GetInputNames();
for (const auto& name : input_names) {
    auto tensor = predictor->GetInputTensor(name);
    tensor->copy(preprocessed_data);
}
```

5) 执行推理

一旦输入数据准备好，就可以执行推理了。这通常涉及调用推理引擎的 Run 方法，该方法会启动模型的推理过程。

```
predictor->Run();
```

6) 获得推理结果并进行后处理

推理执行完成后，首先需要获取输出 Tensor，并将推理结果复制出来。然后，根据模型的输出进行后处理，比如在目标检测任务中，根据检测框的位置裁剪图像。

```
// 获取输出Tensor名称和指针
auto out_names = predictor->GetOutputNames();
std::vector<ProcessedOutput> results;
for (const auto& name : out_names) {
    auto tensor = predictor->GetOutputTensor(name);
    ProcessedOutput data;
    tensor->copy(data);
    results.push_back(processOutput(data));
}

// 后处理，例如裁剪图像等
for (auto& result : results) {
```

```
postprocess(result);  
}
```

小结与思考

- **推理引擎特点**：推理引擎需具备轻量、通用、易用和高性能四大特点，以确保其在多种硬件平台上的高效运行和易部署性。
- **技术挑战**：推理引擎在设计和实现过程中要面临需求复杂性与程序大小的权衡、算力需求与资源碎片化的矛盾，以及执行效率与模型精度的双重要求等挑战。
- **整体架构**：推理引擎的整体架构包括优化阶段和运行阶段，涉及模型转换工具、模型压缩、端侧学习、性能对比与集成模块、中间表示和 Runtime 等关键组件，共同确保模型的高效推理。

14.6 昇腾推理引擎 MindIE

本节将介绍华为昇腾推理引擎 MindIE 的详细内容，包括其基本介绍、关键功能特性以及不同组件的详细描述。

本节内容将深入探讨 MindIE 的三个主要组件：MindIE-Service、MindIE-Torch 和 MindIE-RT，以及它们在服务化部署、大模型推理和推理运行时的功能特性和应用场景。通过本节的介绍，读者将对 MindIE 有一个全面了解，包括其如何支持 AI 业务高效运行和模型快速部署。

14.6.1 MindIE 基本介绍

MindIE（Mind Inference Engine，昇腾推理引擎）是华为昇腾针对 AI 全场景业务的推理加速套件。通过分层开放 AI 能力，支撑用户多样化的 AI 业务需求，使能百模千态，释放昇腾硬件设备算力。支持多种主流 AI 框架，提供多层次编程接口，帮助用户快速构建基于昇腾平台的推理业务。

业界标准 RPC 接口高效对接业务层，支持 Triton 和 TGI 等主流推理服务框架，实现小时级应用部署。提供针对 LLM（Transformer）和文生图（SD 模型）的加速参考代码和预置模型，开箱性能业界领先。少量代码实现训练向推理平滑迁移。

昇腾推理引擎支持请求并发调度和模型多实例并发调度，支持多种异步下发，多流水执行，实现高效的推理加速。支持从 PyTorch 和昇思对接从训练模型转换推理模型的过程，支持多种推理服务框架和兼容接口。提供基于昇腾架构亲和加速技术，覆盖推理全流程的图转换、组网、编译、推理执行、调试调优接口。

已发布 MindIE-Service、MindIE-Torch、MindIE-RT 三个组件。

1. MindIE-Service

MindIE-Service 针对通用模型的推理服务化场景，实现开放、可扩展的推理服务化平台架构，支持对接业界主流推理框架接口，满足大语言模型、文生图等多类型模型的高性能推理需求。

MindIE-Server 作为推理服务端，提供模型服务化能力；MindIE-Client 提供服务客户端标准 API，简化用户服务调用。MindIE-Service 向下调用了 MindIE-LLM 组件能力。

2. MindIE-Torch

MindIE-Torch 是针对 PyTorch 框架模型的推理加速插件。PyTorch 框架上训练的模型利用 MindIE-Torch 提供的简易 C++/Python 接口，只需少量代码即可完成模型迁移，实现高性能推理。MindIE-Torch 向下调用了 MindIE-RT 组件能力。

3. MindIE-RT

MindIE-RT 是面向昇腾 AI 处理器的推理加速引擎，提供模型推理迁移相关开发接口及工具，能够将不同的 AI 框架（PyTorch、ONNX 等）上完成训练的算法模型统一为计算图表示，具备多粒度模型优化、整图下发以及推理部署等功能。集成 Transformer 高性能算子加速库 ATB，提供基础高性能算子和高效的算子组合技术（Graph），便于模型加速。

14.6.2 关键功能特性

1. 服务化部署

MindIE-Service 的组件包括 MindIE-Server、MindIE-Client、Benchmark 评测工具等，一方面通过对接昇腾的推理加速引擎，带来大模型在昇腾环境中的性能提升；另一方面，通过接入现有的主流推理框架生态，逐渐以性能和易用性牵引存量生态的用户向全自研推理服务化平台迁移。

支持的特性：

- 支持大模型服务化快速部署。
- 提供了标准的昇腾服务化接口，兼容 Triton/OpenAI/TGI/vLLM 等第三方框架接口。
- 支持 Continuous Batching、PagedAttention。
- 支持基于 Transformer 推理加速库（Ascend Transformer Boost）的模型接入，继承其加速能力，包括融合加速算子、量化等特性。

2. 大模型推理

提供大模型推理能力，支持大模型业务全流程、逐级能力开放，使能大模型客户需求定制化。

1) PyTorch 模型迁移

对接主流 PyTorch 框架，实现训练到推理的平滑迁移，提供通用的图优化并行推理能力，为用户提供深度定制优化能力。

2) MindIE-Torch TorchScript 支持以下功能特性

- 支持 TorchScript 模型的编译优化，生成可直接在昇腾 NPU 设备加速推理的 TorchScript

模型。

- 支持静态输入和动态输入，动态输入分为动态 Dims 和 ShapeRange 两种模式。
- 编译优化时支持混合精度、FP32 以及 FP16 精度策略。
- 支持用户自定义 Converter 和自定义 Pass。
- 支持异步推理和异步数据拷贝。
- 支持与 torch_npu 配套使用，算子可 fallback 到 torch_npu 执行。
- 支持多语言 API（C++、Python）。

3) MindIE-Torch ExportedProgram 支持以下功能特性

● 支持 ExportedProgram 的编译优化，生成可直接在昇腾 NPU 设备加速推理的 nn.Module 模型。

- 支持静态输入和动态 ShapeRange 输入。
- 编译优化时支持混合精度、FP32 以及 FP16 精度策略。
- 支持异步推理和异步数据拷贝。
- 支持 Python API。

3. 推理运行时

集成推理应用接口及 Transformer 加速库，提供推理迁移相关开发接口及工具，提供通用优化及并行推理能力。

MindIE-RT 集成昇腾高性能算子加速库 ATB，为实现基于 Transformer 的神经网络推理加速引擎库，库中包含了各类 Transformer 类模型的高度优化模块，如 Encoder 和 Decoder 部分。

MindIE-RT 专注于为用户提供快速迁移、稳定精度以及极致性能的推理服务，让用户能够脱离底层硬件细节和不同平台框架的差异，专注于推理业务本身，实现高效的模型部署开发。并且专门针对大模型下的 Transformer 架构，提高 Transformer 模型性能，提供了基础的高性能的算子、高效的算子组合技术（Graph），方便模型加速。目前 MindIE-RT 已实现动态输入推理，解析框架模型等功能特性。

1) MindIE-RT 支持以下功能特性

- 支持多语言 API（C++、Python），详情参见 C++编程模型和 Python 编程模型。
- 提供 parser，支持直接导入人工智能框架 ONNX 模型，详情参见解析框架模型。
- 支持 Transformer 算子加速库，集成基础高性能算子，详情可见 ATB 高性能加速库使用。
- 支持丰富的编译时优化方法和运行时优化方法，用户可以在昇腾 AI 处理器上占用较少的内存，并部署更高性能的推理业务，提供的优化方法有精度优化和常量折叠。

2) 应用场景

MindIE-RT 是基于昇腾 AI 处理器的部署推理引擎，适用于通过 NPU、GPU、CPU 等设备训练的算法模型，为其提供极简易用且灵活的接口，实现算法从训练到推理的快速迁移。目前 MindIE-RT 的快速迁移能力已支持以下 4 类业务场景：

- 计算机视觉。
- 自然语言处理。
- 推荐检索。
- 大模型对话。

小结与思考

- **MindIE 概述：**MindIE 是华为昇腾推出的 AI 推理加速套件，提供多层次编程接口和支持多种 AI 框架，实现高效推理加速和快速模型部署。
- **关键功能特性：**MindIE 具备服务化部署、大模型推理能力，以及集成推理应用接口和 Transformer 加速库，支持多语言 API，优化模型迁移和推理性能。
- **应用场景：**MindIE 适用于计算机视觉、自然语言处理、推荐检索以及大模型对话等多种 AI 应用场景，提供从训练到推理的快速迁移和高性能部署能力。

14.7 昇腾计算语言 AscendCL

AscendCL 作为华为 Ascend 系列 AI 处理器的软件开发框架，为用户提供了强大的编程支持。通过 AscendCL，开发者可以更加高效地进行 AI 应用的开发和优化，从而加速 AI 技术各个领域的应用和落地。AscendCL 的易用性和高效性，使得它成为开发 AI 应用的重要工具之一。

本节将介绍 AscendCL 的概念、优势、应用场景以及基本开发流程。

14.7.1 AscendCL 基本介绍

AscendCL（Ascend Computing Language）是一套用于在昇腾平台上开发深度神经网络应用的 API 库，提供运行资源管理、内存管理、模型加载与执行、算子加载与执行、媒体数据处理等 API，能够实现利用昇腾硬件计算资源，在昇腾 CANN 平台上进行深度学习推理计算、图形图像预处理、单算子加速计算等。

简单来说，就是统一的 API 框架，实现对所有资源的调用。其中，计算资源层是昇腾 AI 处理器的硬件算力基础，主要完成神经网络的矩阵相关计算，完成控制算子、标量、向量等通用计算和执行控制功能，完成图像和视频数据的预处理，为深度神经网络计算提供了执行上的保障。

1. AscendCL 优势

AscendCL 优势主要有三个要点：

- **高度抽象：**算子编译、加载、执行的 API 归一，相比每个算子一个 API，AscendCL 大幅减少 API 数量，降低复杂度。
- **向后兼容：**AscendCL 具备向后兼容，确保软件升级后，基于旧版本编译的程序依然可以

在新版本上运行。

- 零感知芯片：一套 AscendCL 接口可以实现应用代码统一，多款昇腾 AI 处理器无差异。

2. AscendCL 应用场景

如图 14.7.1 所示，AscendCL 应用场景及调用如下：

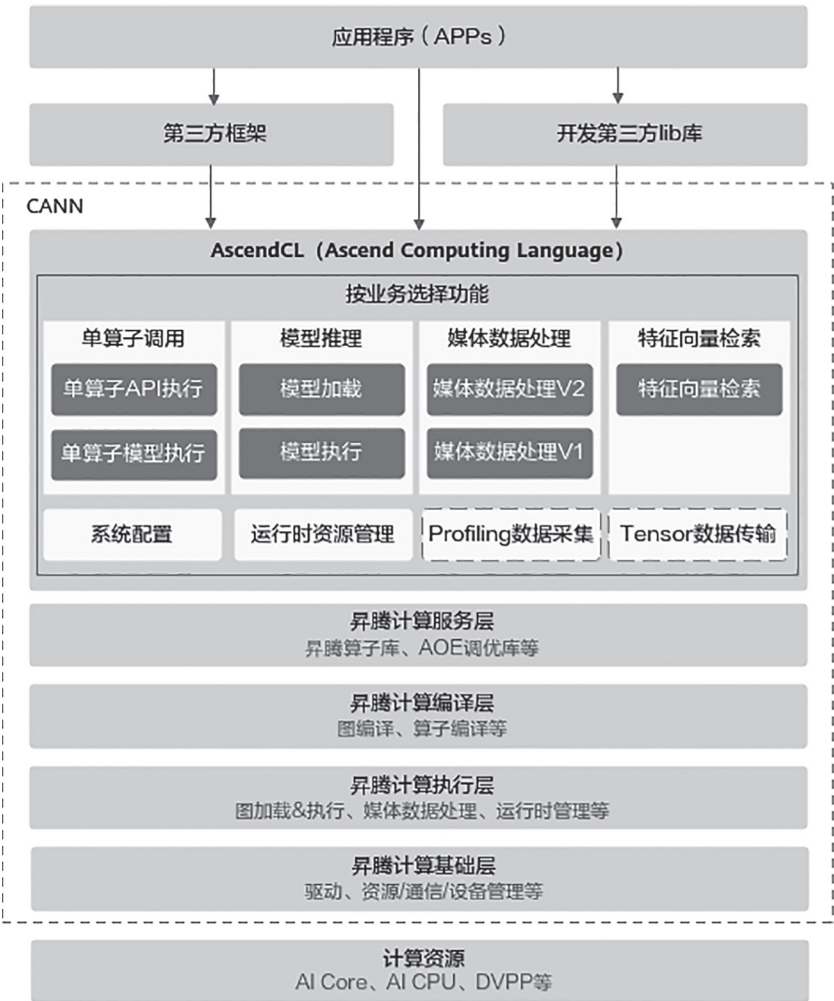


图 14.7.1

- 开发应用：用户可以直接调用 AscendCL 提供的接口开发图片分类应用、目标识别应用等。
- 供第三方框架调用：用户可以通过第三方框架调用 AscendCL 接口，以便使用昇腾 AI 处理器的计算能力。
- 供第三方开发 lib 库：用户还可以使用 AscendCL 封装实现第三方 lib 库，以便提供昇腾 AI 处理器的运行管理、资源管理等能力。

14.7.2 基本概念

AscendCL 基本概念如表 14.7.1 所示。

表 14.7.1 AscendCL 基本概念及描述

概念	描述
异步/同步	本书提及的异步、同步是站在调用者和执行者的角度。若在调用 AscendCL 接口后 不等待 Device 侧的任务执行完成再返回，则表示调度是 异步 的；若在调用 AscendCL 接口后 需等待 Device 侧的任务执行完成再返回，则表示调度是 同步 的
进程/线程	本书提及的进程、线程，若无特别说明，则表示 Host 上的进程、线程
Host	Host 指与 Device 相连接的 x86 服务器、ARM 服务器，会利用 Device 提供的 NN 计算能力，完成业务
Device	Device 指安装了昇腾 AI 处理器的硬件设备，利用 PCIe 接口与 Host 侧连接，为 Host 提供 NN 计算能力。若存在多个 Device，多个 Device 之间的内存资源不能共享
Context	Context 作为一个容器，管理了所有对象（包括 Stream、Event、设备内存等）的生命周期。不同 Context 的 Stream，不同 Context 的 Event 是完全隔离的，无法建立同步等待关系。Context 分为两种：①默认 Context，调用 aclrtSetDevice 接口指定用于运算的 Device 时，系统会自动隐式创建一个默认 Context，一个 Device 对应一个默认 Context，默认 Context 不能通过 aclrtDestroyContext 接口来释放。②显式创建 Context， 推荐 ，在进程或线程中调用 aclrtCreateContext 接口显式创建一个 Context
Stream	Stream 用于维护一些异步操作的执行顺序，确保按照应用程序中的代码调用顺序在 Device 上执行。基于 Stream 的 Kernel 执行和数据传输能够实现 Host 运算操作、Host 与 Device 间的数据传输、Device 内的运算并行。Stream 分两种：①默认 Stream，调用 aclrtSetDevice 接口指定用于运算的 Device 时，系统会自动隐式创建一个默认 Stream，一个 Device 对应一个默认 Stream，默认 Stream 不能通过 aclrtDestroyStream 接口来释放。②显式创建 Stream， 推荐 ，在进程或线程中调用 aclrtCreateStream 接口显式创建一个 Stream
Event	支持调用 AscendCL 接口同步 Stream 之间的任务，例如同一个 Device 上的多个任务。例如，若 Stream2 的任务依赖 Stream1 的任务，想保证 Stream1 中的任务先完成，这时可创建一个 Event，并将 Event 插入到 Stream1，在执行 Stream2 的任务前，先同步等待 Event 完成

续表

概念	描述
AIPP	AIPP（Artificial Intelligence Pre-Processing）用于在 AI Core 上完成图像预处理，包括色域转换（转换图像格式）、图像归一化（减均值/乘系数）和抠图（指定抠图起始点，抠出神经网络需要大小的图片）等。AIPP 区分为静态 AIPP 和动态 AIPP。只能选择静态 AIPP 或动态 AIPP 方式来处理图片，不能同时配置静态 AIPP 和动态 AIPP 两种方式。①静态 AIPP，模型转换时设置 AIPP 模式为静态，同时设置 AIPP 参数，模型生成后，AIPP 参数值被保存在离线模型（*.om），每次模型推理过程采用固定的 AIPP 预处理参数（无法修改）。如果使用静态 AIPP 方式，多 Batch 情况下共用同一份 AIPP 参数。②动态 AIPP，模型转换时设置 AIPP 模式为动态，每次模型推理前，根据需求，在执行模型前设置动态 AIPP 参数值，然后在模型执行时可使用不同的 AIPP 参数。如果使用动态 AIPP 方式，多 Batch 可使用不同的 AIPP 参数
动态 Batch/动态分辨率	在某些场景下，模型每次输入的 Batch Size 或分辨率是不固定的，如检测出目标后再执行目标识别网络，由于目标个数不固定，目标识别网络输入 Batch Size 不固定。①动态 Batch，用户执行推理时，其 Batch Size 是动态可变的。②动态分辨率，用户执行推理时，每张图片的分辨率 $H*W$ 是动态可变的
动态维度（ND 格式）	为了支持 Transformer 等网络在输入格式的维度不确定的场景，需要支持 ND 格式下任意维度的动态设置。ND 表示支持任意格式，当前 $N \leq 4$
通道	在 RGB 色彩模式下，图像通道就是指单独的红色 R、绿色 G、蓝色 B 部分。也就是说，一幅完整的图像由红色、绿色、蓝色三个通道组成，它们共同作用产生了完整的图像。同样在 HSV 色系中指的是色调 H、饱和度 S、亮度 V 三个通道
标准形态	指 Device 作为 EP，通过 PCIe 配合主设备（x86、ARM 等各种服务器）进行工作，此时 Device 上的 CPU 资源仅能通过 Host 调用，相关推理应用程序运行在 Host。Device 只为服务器提供 NN 计算能力
EP 模式	以昇腾 AI 处理器的 PCIe 的工作模式进行区分，如果 PCIe 工作在从模式，则称为 EP 模式
RC 模式	以昇腾 AI 处理器的 PCIe 的工作模式进行区分，如果 PCIe 工作在主模式，可以扩展外设，则称为 RC 模式

1. Device、Context、Stream 关系

下面将介绍 Device、Context 和 Stream 在 AscendCL 框架中的作用及其相互关系，也简单介绍 Task/Kernel（图 14.7.2）。

1) Device

Device 用于指定计算设备。

Device 的生命周期源于首次调用 aclrtSetDevice 接口。每次调用 aclrtSetDevice 接口，系统会进行引用计数加 1；调用 aclrtResetDevice 接口，系统会进行引用计数减 1。当引用计数减为 0 时，在本进程中 Device 上的资源不可用。

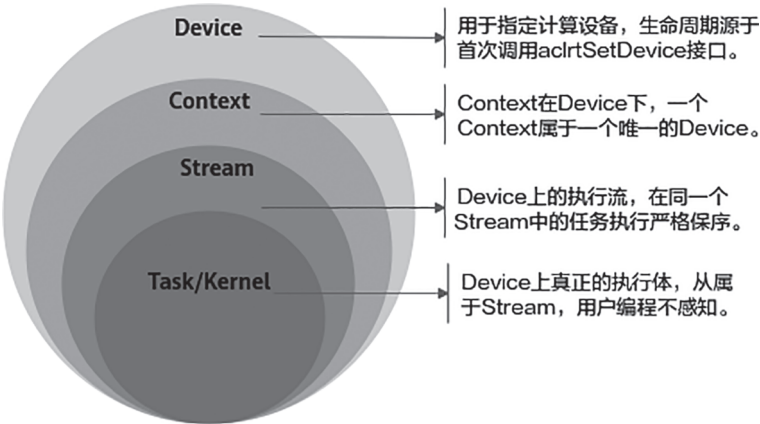


图 14.7.2

2) Context

在 Device 下，一个 Context 一定属于一个唯一的 Device。

Context 分隐式创建和显式创建。隐式创建的 Context（即默认 Context），生命周期始于调用 [aclrtSetDevice](#) 接口，终结于调用 [aclrtResetDevice](#) 接口使引用计数为零时。隐式 Context 只会被创建一次，调用 [aclrtSetDevice](#) 接口重复指定同一个 Device，只增加隐式创建的 Context 的引用计数。

显式创建的 Context，生命周期始于调用 [aclrtCreateContext](#) 接口，终结于调用 [aclrtDestroyContext](#) 接口。若在某一进程内创建多个 Context（Context 的数量与 Stream 相关，Stream 数量有限制，请参见 [aclrtCreateStream](#)），当前线程在同一时刻内只能使用其中一个 Context，建议通过 [aclrtSetCurrentContext](#) 接口明确指定当前线程的 Context，增加程序的可维护性。

值得注意的是，进程内的 Context 是共享的，可以通过 [aclrtSetCurrentContext](#) 进行切换。

3) Stream

Stream 是 Device 上的执行流，在同一个 Stream 中的任务执行严格保序。

Stream 分隐式创建和显式创建。每个 Context 都会包含一个默认 Stream，这个属于隐式创建，隐式创建的 Stream 生命周期同归属的 Context。

用户可以显式创建 Stream，显式创建的 Stream 生命周期始于调用 [aclrtCreateStream](#)，终结于调用 [aclrtDestroyStream](#) 接口。显式创建的 Stream 归属的 Context 被销毁或生命周期结束后，会影响该 Stream 的使用，虽然此时 Stream 没有被销毁，但不可再用。

4) Task/Kernel

Task/Kernel 是 Device 上真正的任务执行体。

2. 线程、Context、Stream 关系

一个用户线程一定会绑定一个 Context，所有 Device 的资源使用或调度都必须基于 Context。一个线程中当前会有一个唯一的 Context 在用，Context 中已经关联了本线程要使用的 Device。此时可以通过 `aclrtSetCurrentContext` 进行 Device 的快速切换。

一个线程中可以创建多个 Stream，不同的 Stream 上计算任务可以并行执行；多线程场景下，推荐每个线程创建一个 Stream，线程之间的 Stream 在 Device 上相互独立，每个 Stream 内部的任务是按照 Stream 下发的顺序执行。

多线程的调度依赖于运行应用的操作系统调度，多 Stream 在 Device 侧的调度由 Device 上调度组件进行调度。

3. 进程内多线程间 Context 切换

进程内的 Context 工作流如图 14.7.3 所示。

- 一个进程中可以创建多个 Context，但一个线程同一时刻只能使用一个 Context。
- 线程中创建的多个 Context，线程缺省使用最后一次创建的 Context。
- 进程内创建的多个 Context，可以通过 `aclrtSetCurrentContext` 设置当前需要使用的 Context。

4. 默认 Context 和默认 Stream

Device 上执行操作下发前，必须有 Context 和 Stream，这个 Context、Stream 可以显式创建，也可以隐式创建。隐式创建的 Context、Stream 就是默认 Context、默认 Stream。

默认 Context 不允许用户执行 `aclrtGetCurrentContext` 或 `aclrtSetCurrentContext` 操作，也不允许执行 `aclrtDestroyContext` 操作。

默认 Stream 作为接口入参时，直接传 NULL。

默认 Context、默认 Stream 一般适用于简单应用，用户仅仅需要在一个 Device 的计算场景下。多线程应用程序建议全部使用显式创建的 Context 和 Stream。

5. 多线程、多 Stream 性能考虑

线程调度依赖运行的操作系统，Stream 上下发了任务后，Stream 的调度由 Device 的调度单元调度，但如果一个进程内的多 Stream 上的任务在 Device 存在资源争抢的时候，性能可能会比单 Stream 低。

当前昇腾 AI 处理器有不同的执行部件，如 AI Core、AI CPU、Vector Core 等，对应使用不同执行部件的任务，建议多 Stream 的创建按照算子执行引擎划分。

单线程、多 Stream 与多线程、多 Stream（一个进程中可以包含多个线程，每个线程中一个 Stream）性能上哪个更优，具体取决于应用本身的逻辑实现，一般来说前者性能略好，原因是相对后者，应用层少了线程调度开销。

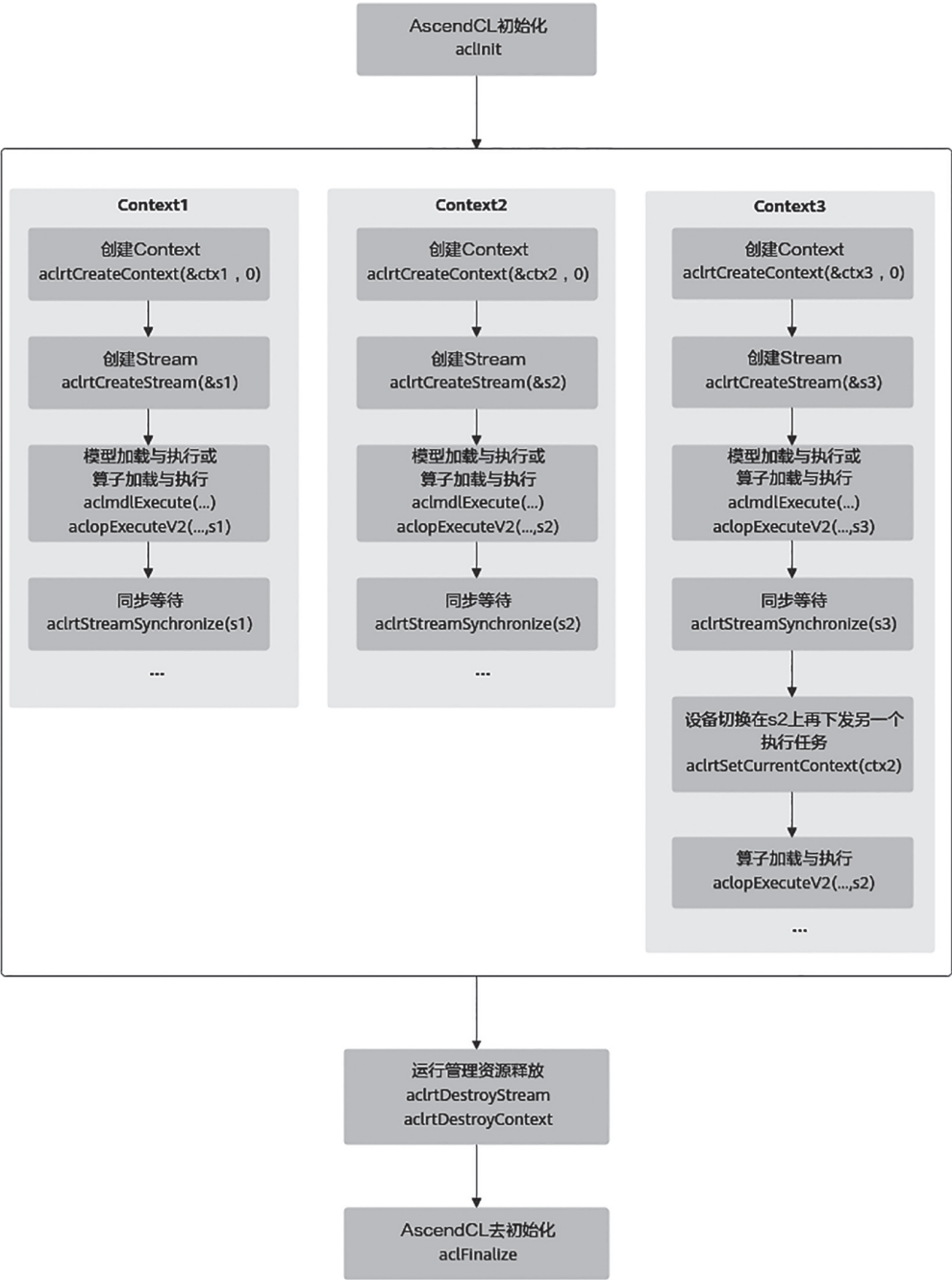
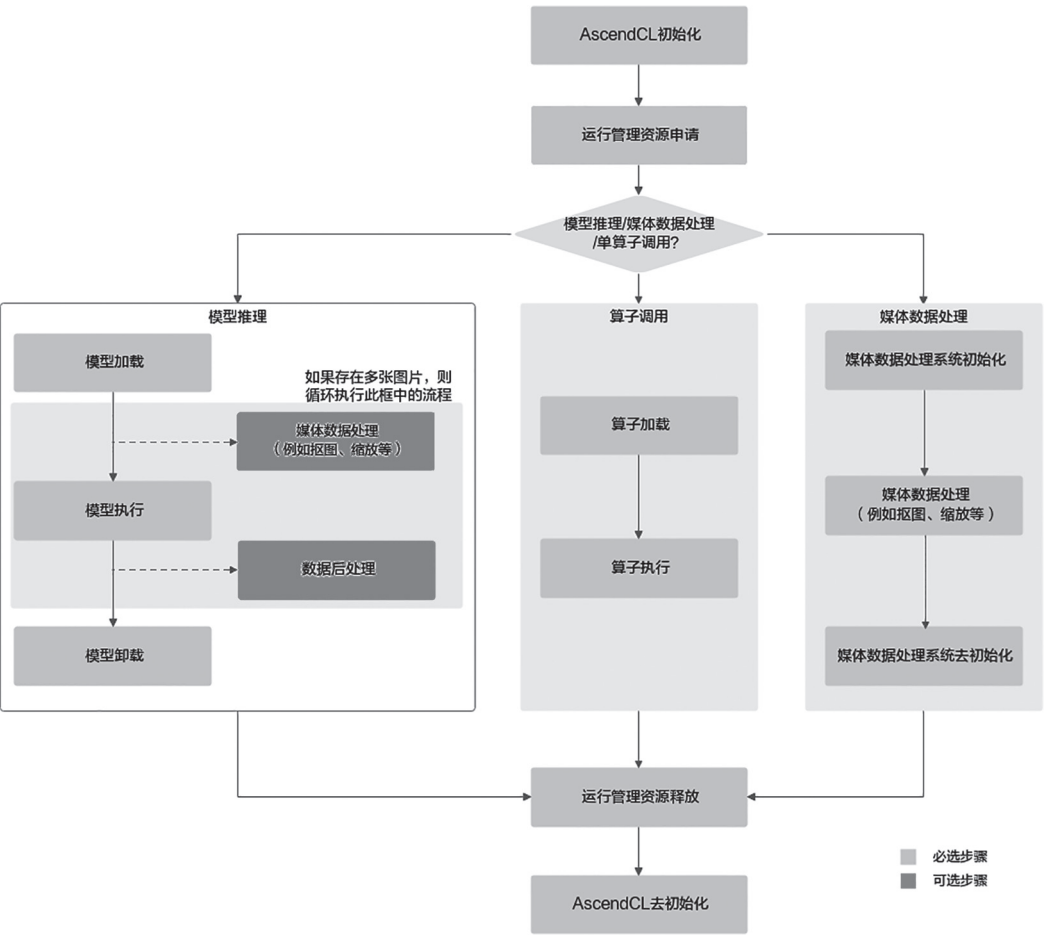


图 14.7.3

14.7.3 基本开发流程

调用 AscendCL 接口，可开发包含模型推理、媒体数据处理、单算子调用等功能的应用，这些功能可以独立存在，也可以组合存在。图 14.7.4 给出了使用 AscendCL 接口开发 AI 应用的整体接口调用流程。

图 14.7.4 根据应用开发中的典型功能抽象出主要的接口调用流程。例如，如果模型对输入图片的宽高要求与用户提供的源图不一致，则需要媒体数据处理，将源图裁剪成符合模型的要求；如果需要实现模型推理的功能，则需要先加载模型，模型推理结束后，则需要卸载模型；如果模型推理后，需要从推理结果中查找最大置信度的类别标识对图片分类，则需要数据后处理。



其基本流程如下所示：

(1) AscendCL 初始化：调用 `aclInit` 接口实现初始化 AscendCL。

(2) 运行管理资源申请：申请运行管理资源（Device、Context、Stream 等）的具体流程。

(3) 具体计算：分为模型推理、单算子调用、媒体数据处理三部分。

①模型推理

- 模型加载：模型推理前，需要先将对应的模型加载到系统中。注意加载模型前需要有适配昇腾 AI 处理器的离线模型。

- （可选）媒体数据处理：可实现 JPEG 图片编/解码、视频解码、抠图/图片缩放/格式转换等功能。

- 模型执行：使用模型实现图片分类、目标识别等推理功能。

- （可选）数据后处理：处理模型推理的结果，此处根据用户的实际需求来处理推理结果，例如用户可以将获取的推理结果写入文件，从推理结果中找到每张图片最大置信度的类别标识等。

- 模型卸载：调用 `aclmdlUnload` 接口卸载模型。

②算子调用

- 如果 AI 应用中不仅仅包括模型推理，还有数学运算（例如 BLAS 基础线性代数运算）、数据类型转换等功能，也想使用昇腾的算力，直接通过 AscendCL 接口加载并执行单个算子，省去模型构建、训练的过程，相对轻量级，又可以使用昇腾的算力。

- 另外，自定义的算子，也可以通过单算子调用的方式来验证算子的功能。

(4) 运行管理资源释放：所有数据处理都结束后，需要依次释放运行管理资源。

(5) AscendCL 去初始化：调用 `aclFinalize` 接口实现 AscendCL 去初始化。

小结与思考

- 高度抽象与兼容性：AscendCL 通过统一的 API 框架简化了算子编译、加载和执行过程，减少了 API 数量，降低了开发复杂度，并确保了向后兼容性，使得基于旧版本编译的程序可以运行在新版本上。
- 多场景应用：AscendCL 支持多种应用场景，包括直接开发图片分类和目标识别等应用，供第三方框架调用以使用昇腾 AI 处理器的计算能力，以及封装实现第三方库以提供运行管理和资源管理能力。
- 资源管理与开发流程：AscendCL 提供详细的资源管理机制，包括 Device、Context 和 Stream 的概念和关系，以及如何显式创建和管理这些资源。基本开发流程涵盖从 AscendCL 初始化、资源申请、模型推理或单算子调用、数据处理到资源释放和去初始化的完整过程。